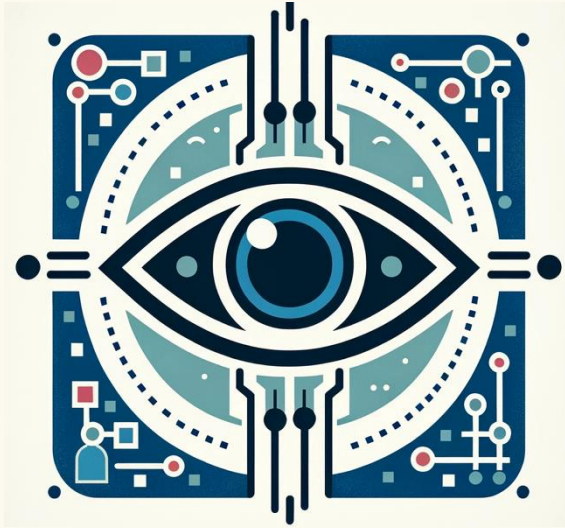




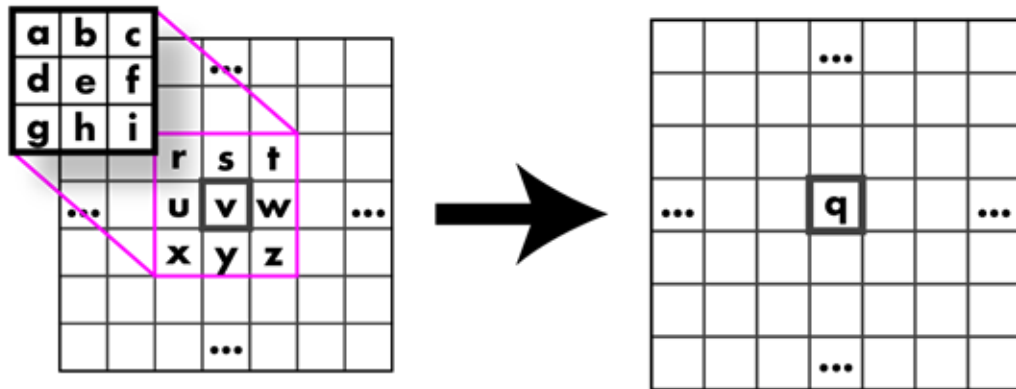
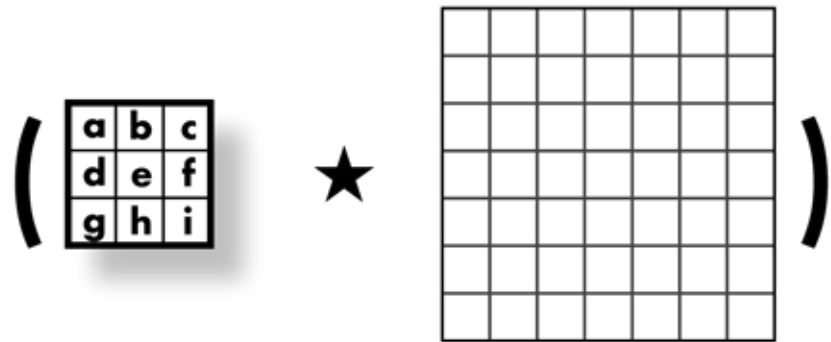
COMPUTER VISION

Features - Edges

Cross-Correlation vs Convolution

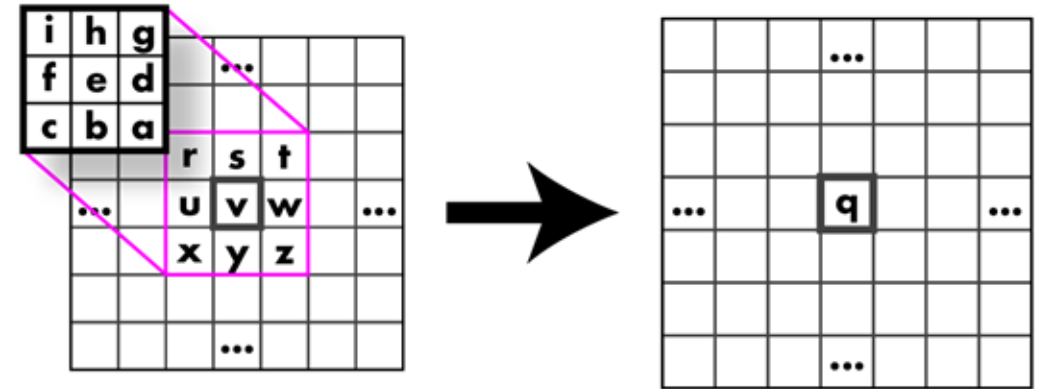
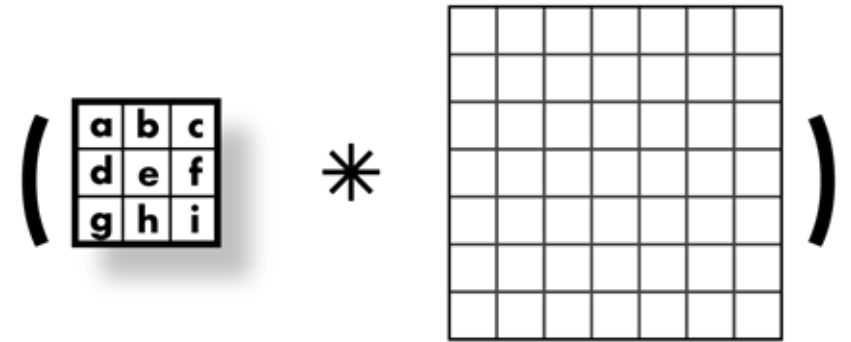
This is what we saw before

Cross-Correlation



$$q = a \times r + b \times s + c \times t + d \times u + e \times v + f \times w + g \times x + h \times y + i \times z$$

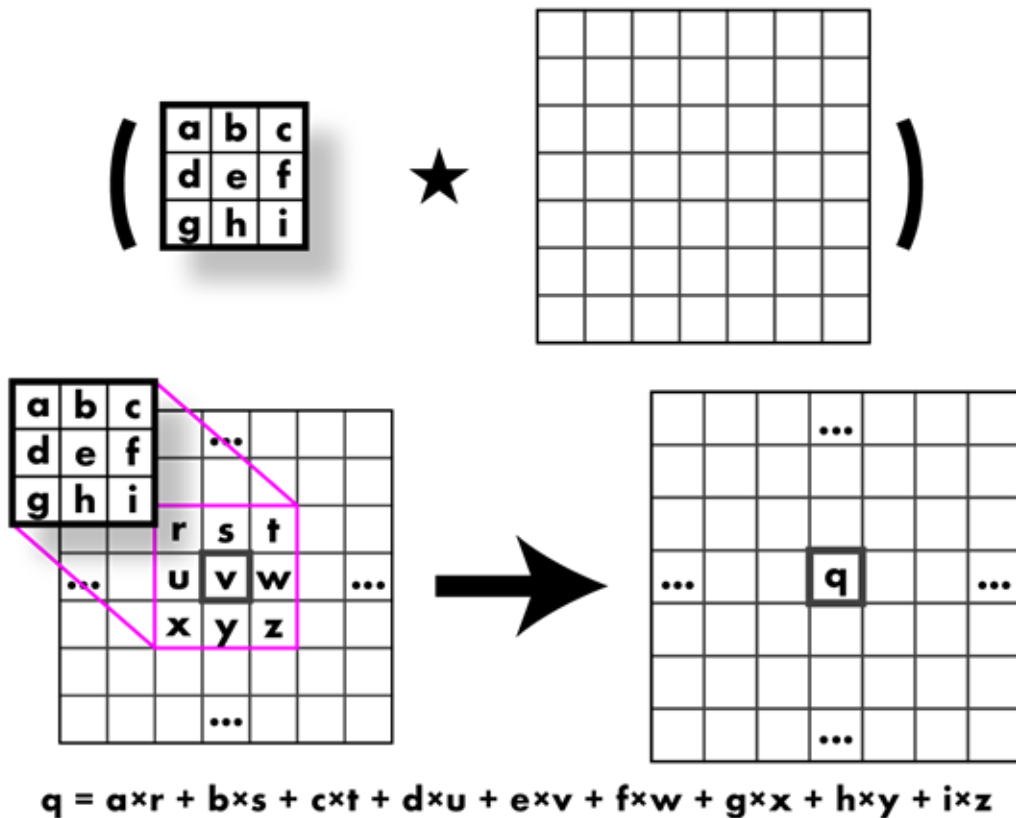
Convolution



$$q = i \times r + h \times s + g \times t + f \times u + e \times v + d \times w + c \times x + b \times y + a \times z$$

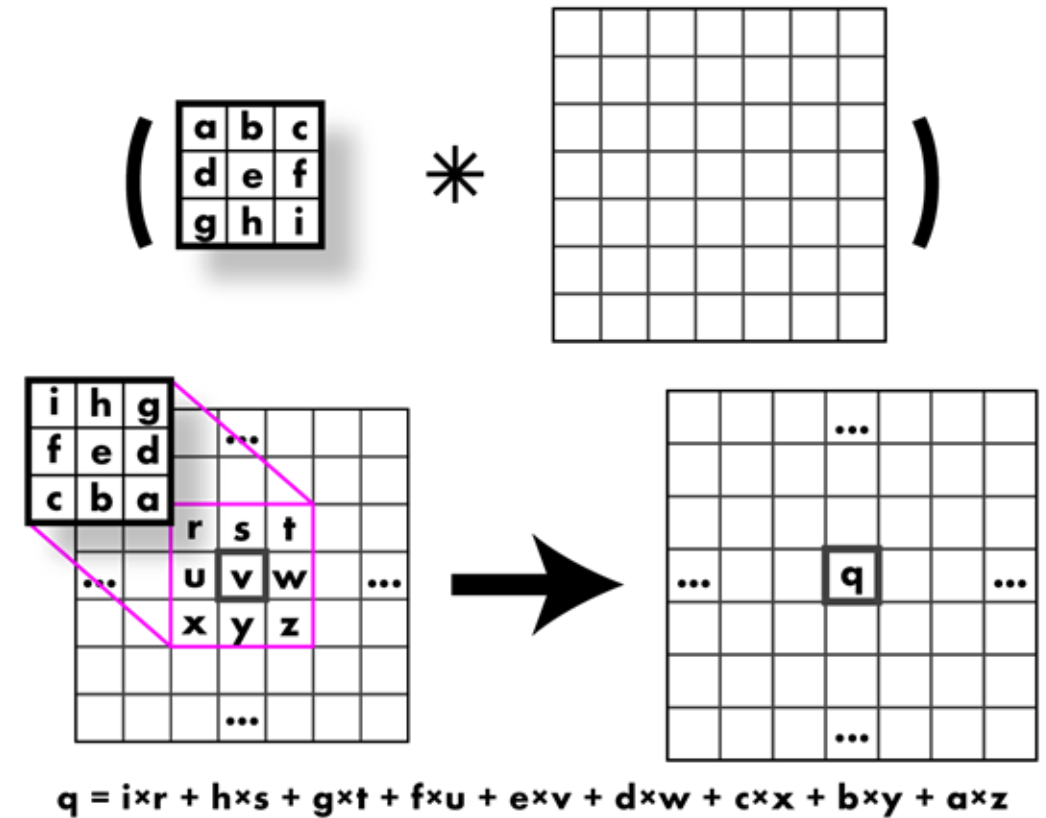
Cross-Correlation vs Convolution

Cross-Correlation



These are nicer mathematically

Convolution



Convolutions - Useful properties

- Commutative
 - $A * B = B * A$
- Associative
 - $A * (B * C) = (A * B) * C$
- Distributes over addition
 - $A * (B + C) = A * B + A * C$
- Plays well with scalars
 - $x(A * B) = (xA) * B = A * (xB)$

Convolutions - Useful properties

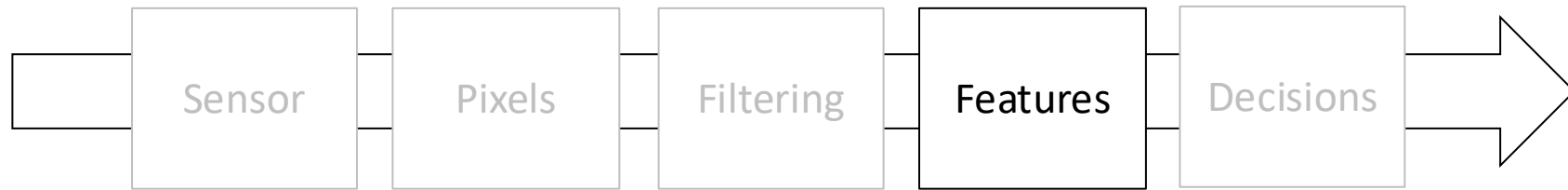
For example, this means some convolutions decompose:

- 2D gaussian is just a composition of 1D gaussians
 - Faster to run two 1D convolutions
 - Especially useful with large filters

Convolutions – What can we do?

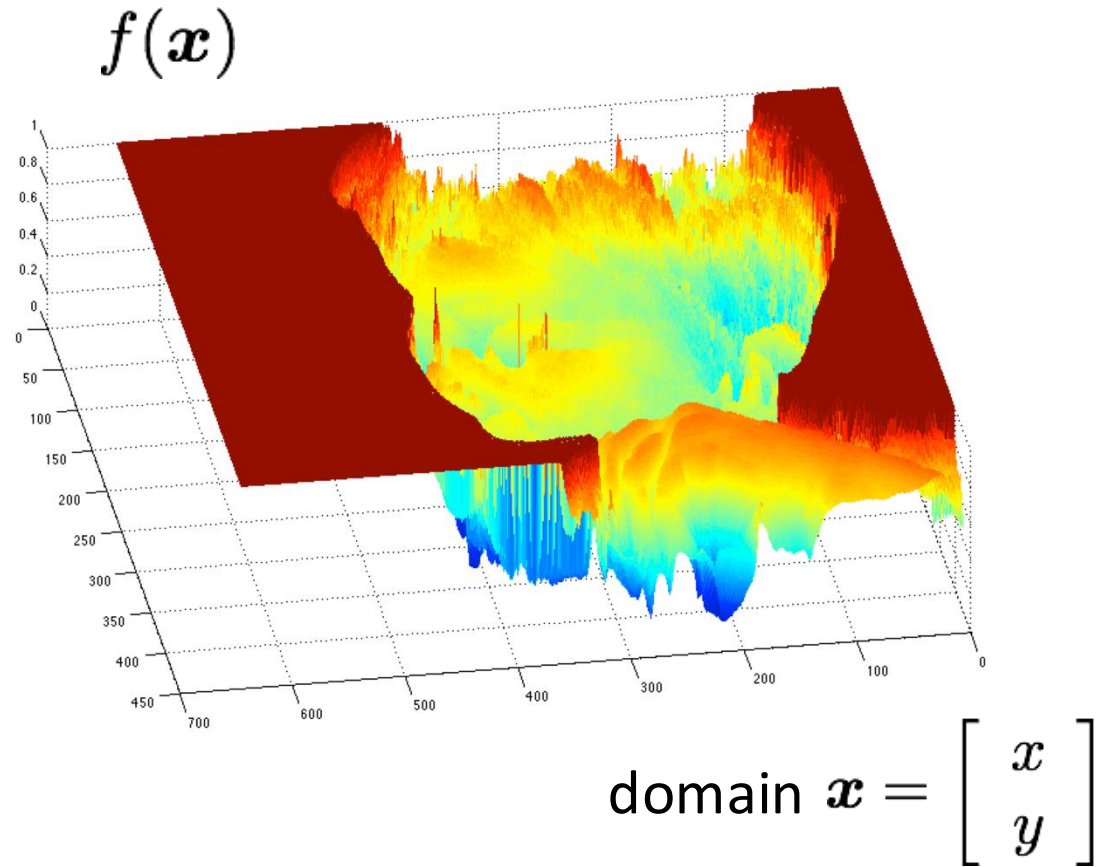
- Blurring
- Sharpening
- Edges
- Features
- Super-resolution
- Classification
- Detection
- Image Generation
- ...

For the most part, computer vision
is based on convolutions



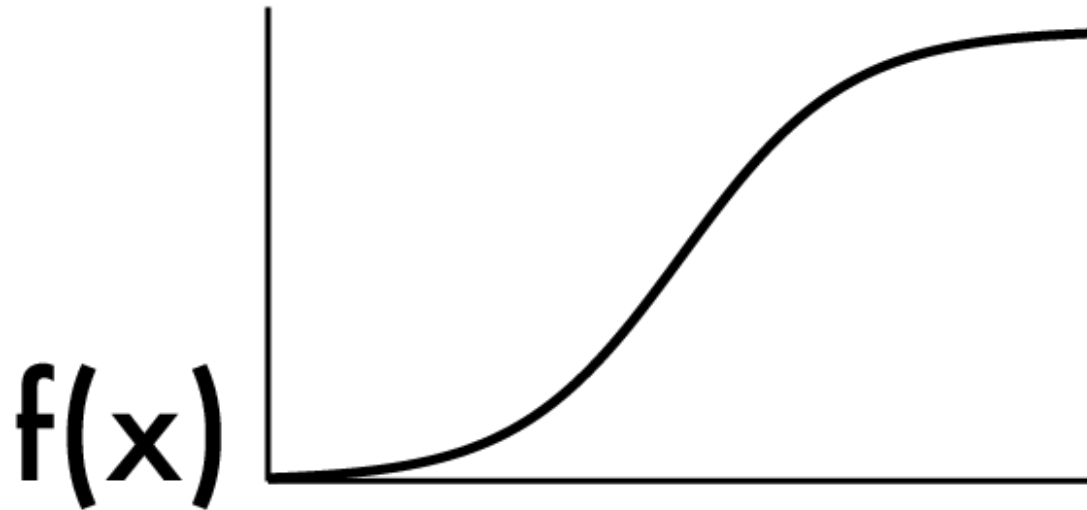
What is an edge?

- If we think of an image as a function



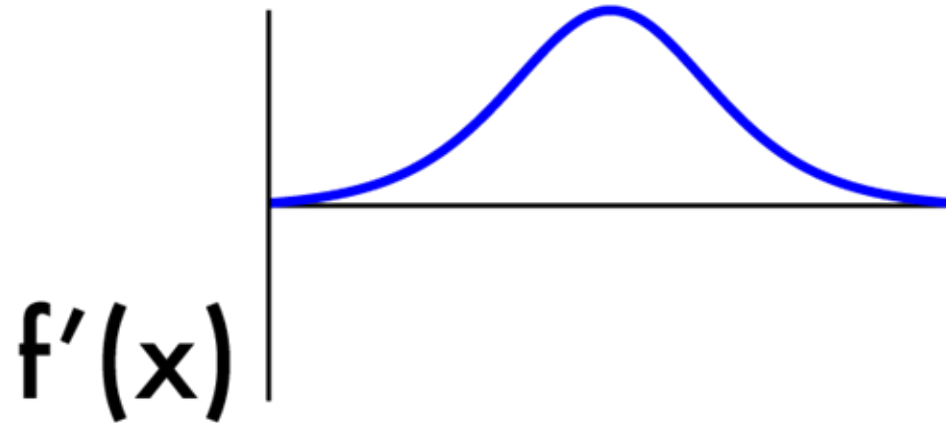
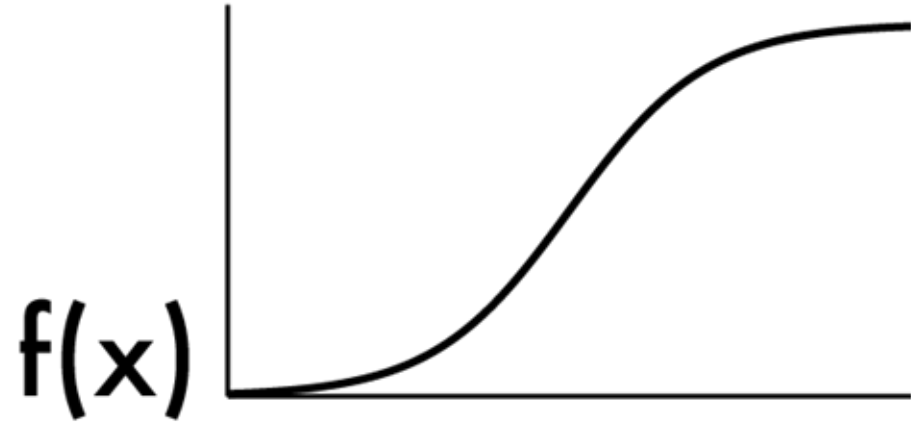
What is an edge?

- If we think of an image as a function
→ Edges are rapid changes in this function



Finding edges

- How can we find edges?
 - Could take derivative
 - Edges = high response

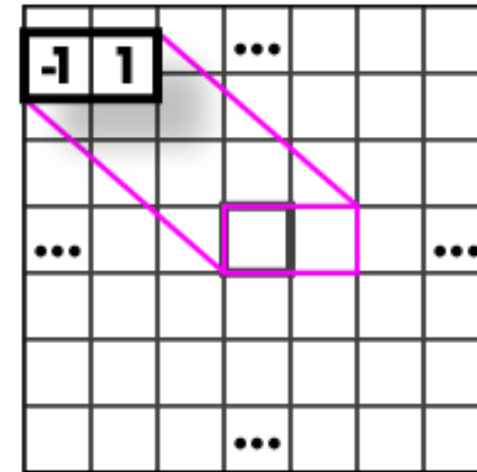
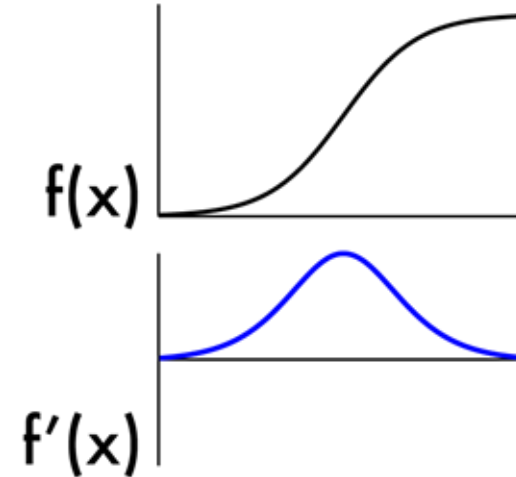


Calculating image derivatives

- We don't have an “actual” function $\rightarrow$ we need to estimate
- Recall: (limit definition of the derivative at a point)

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

- Possibility: set $h = 1$
- If we think of a filter, what will that look like?



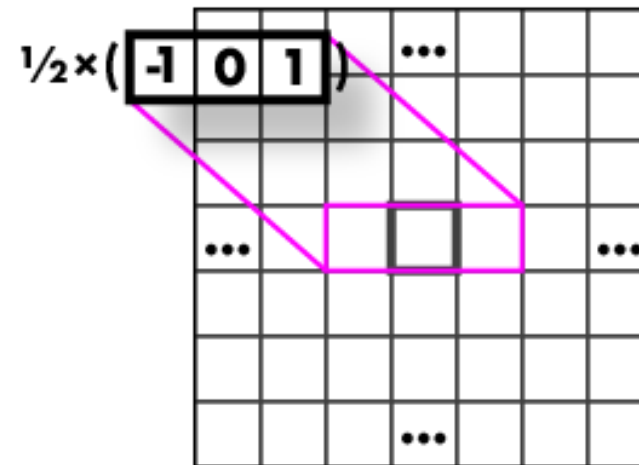
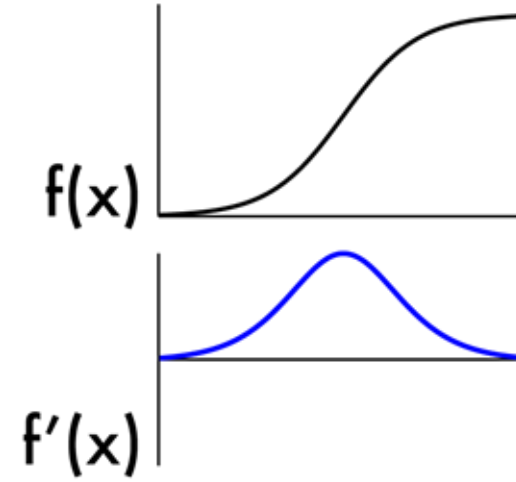
Calculating image derivatives

- We don't have an "actual" Function $\rightarrow$ we need to estimate

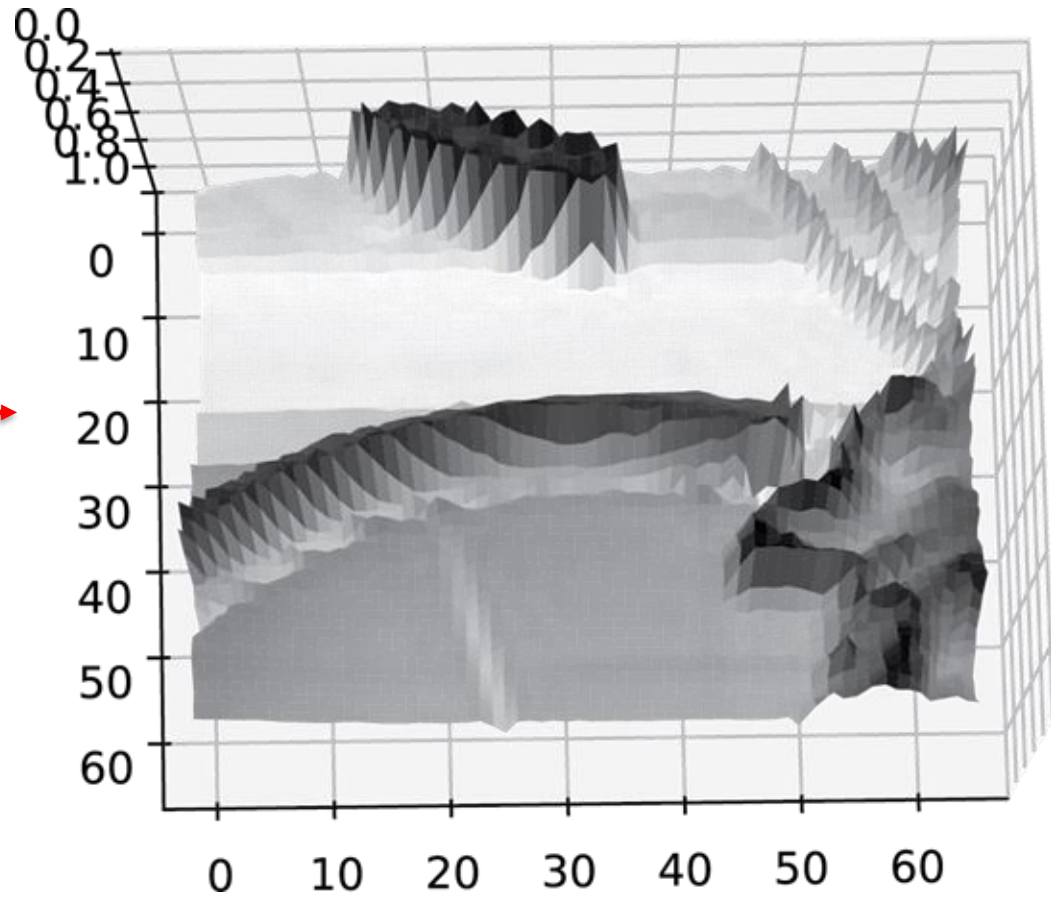
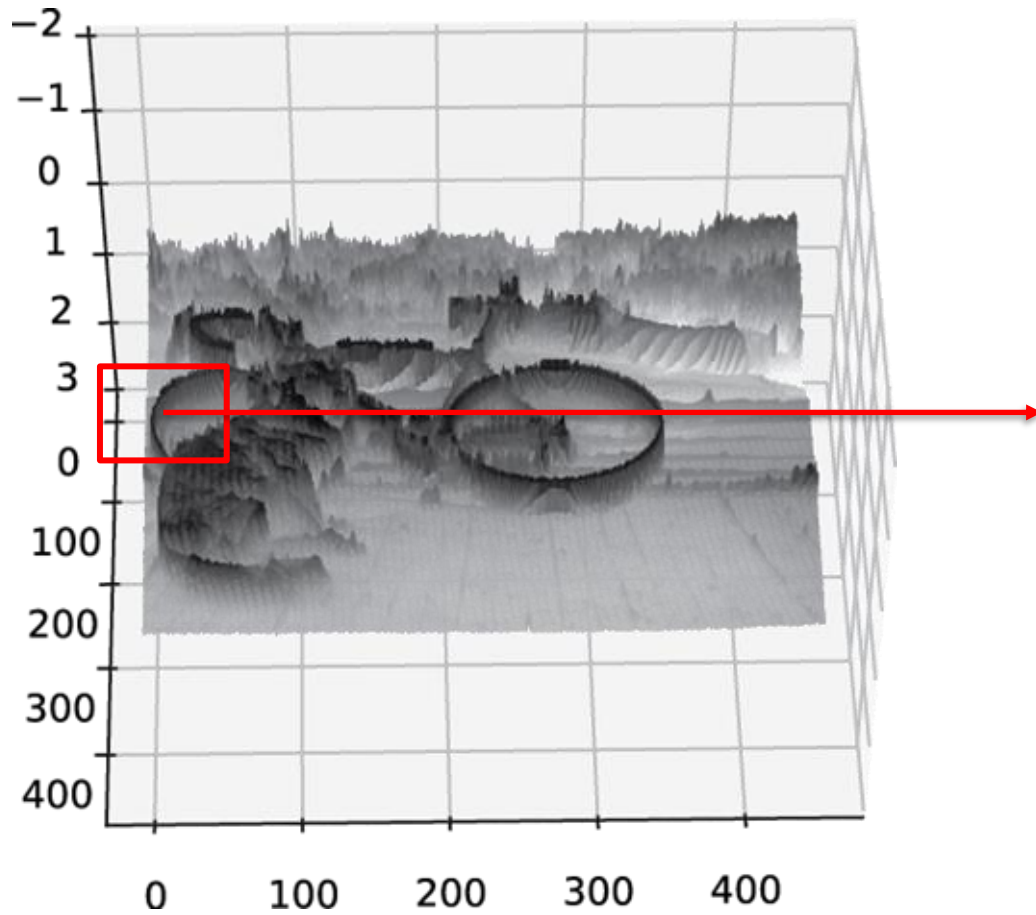
- Recall:

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

- Possibility: set $h = 2$
- If we think of a filter, what will that look like?



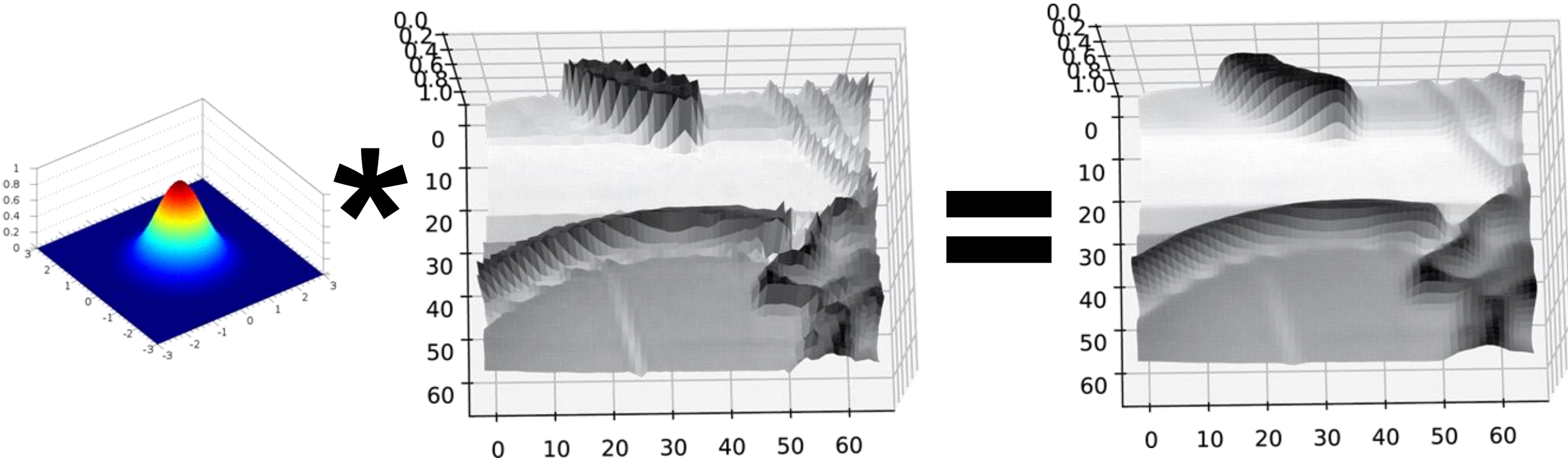
Calculating image derivatives



Images are noisy

Calculating image derivatives

But we already know how to smooth



Calculating image derivatives

Smooth first, then calculate derivative

$$\frac{1}{2} \times \begin{pmatrix} -1 & 0 & 1 \end{pmatrix} * \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} * \begin{pmatrix} \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \\ \square & \square & \square & \square & \square & \square & \square \end{pmatrix}$$

Calculating image derivatives

Smooth first, then calculate derivative

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right) * \begin{bmatrix} & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \end{bmatrix}$$

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$

-1	0	1
-----------	----------	----------

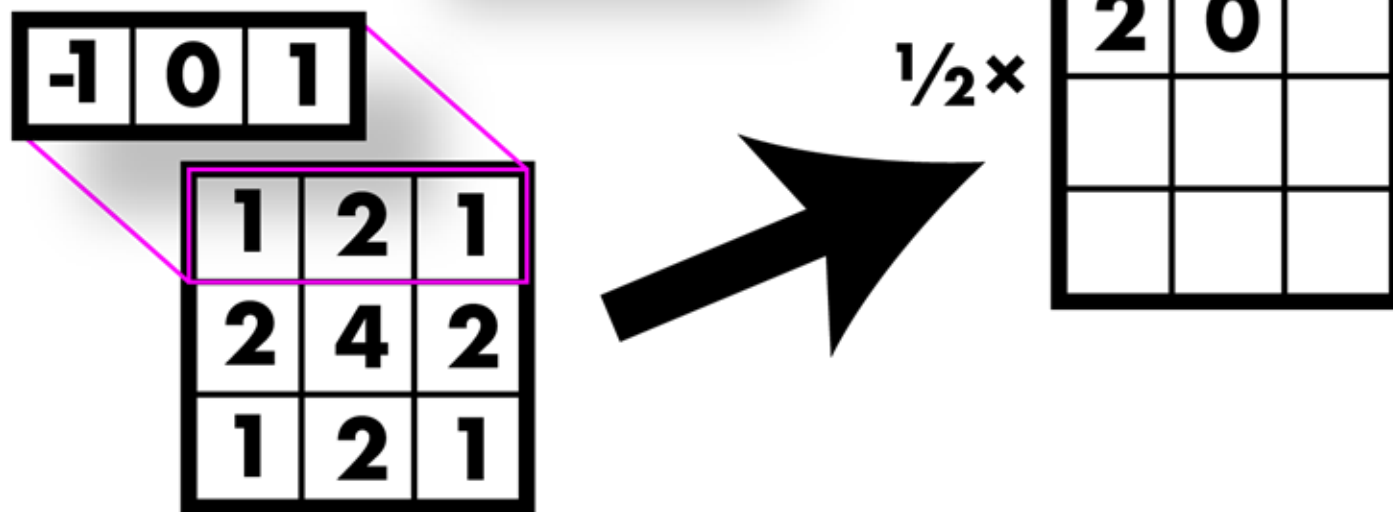
1	2	1
2	4	2
1	2	1



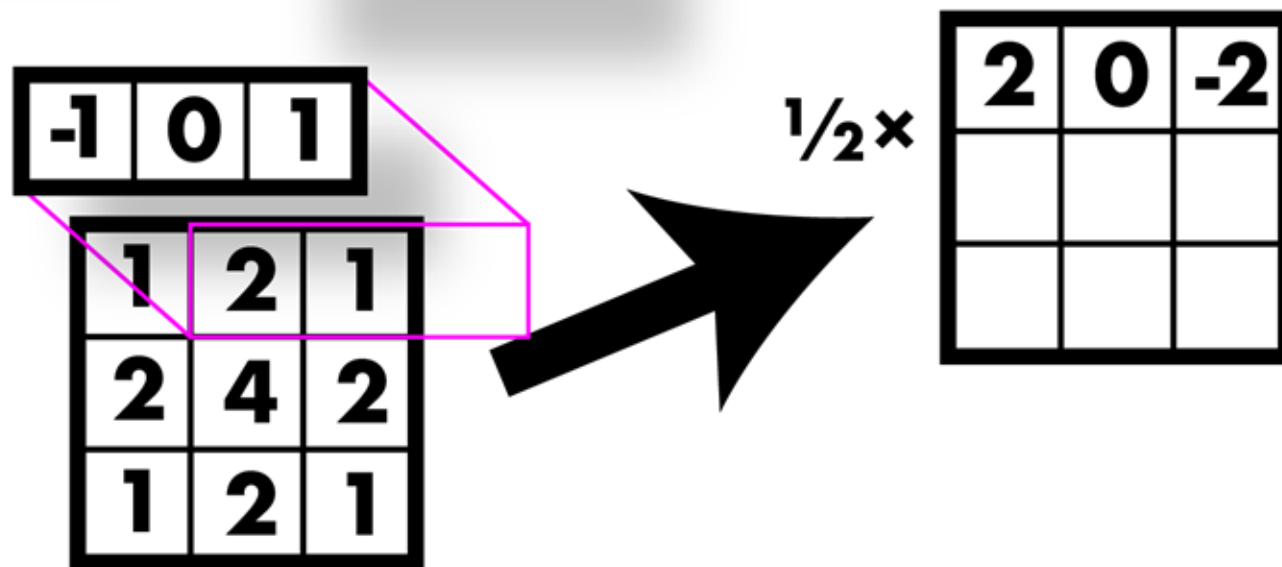
$\frac{1}{2} \times$

2		

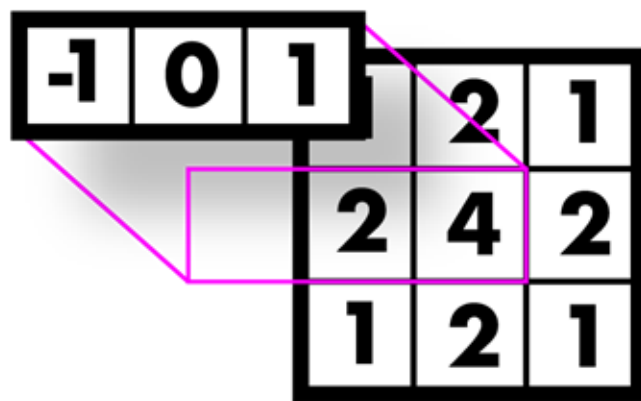
$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$

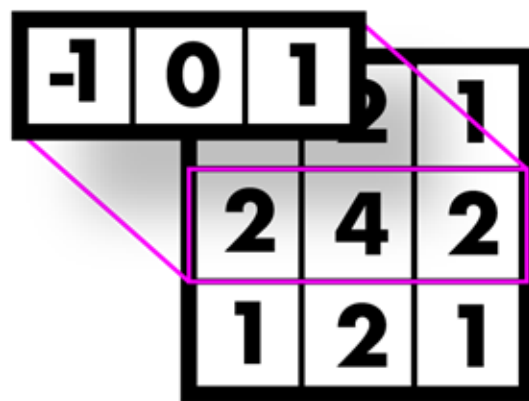


$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



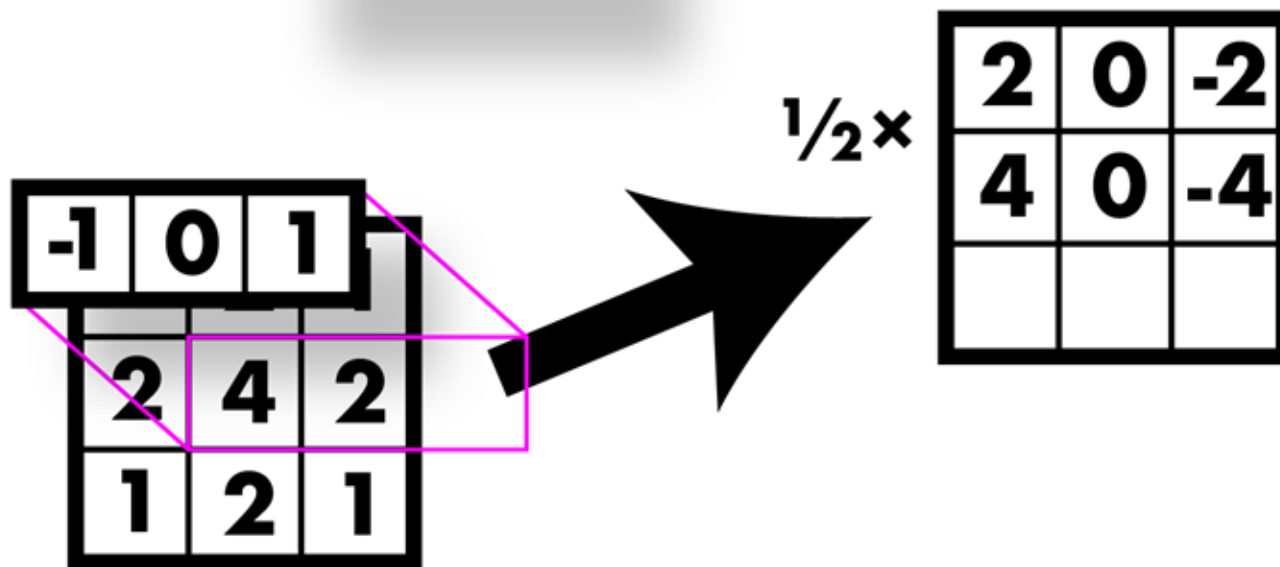
$$\frac{1}{2} \times \begin{bmatrix} 2 & 0 & -2 \\ 4 & & \\ & & \end{bmatrix}$$

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$

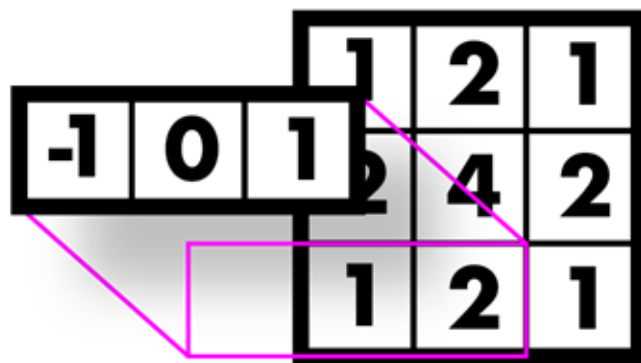


$$\frac{1}{2} \times \begin{bmatrix} 2 & 0 & -2 \\ 4 & 0 & \\ & & \end{bmatrix}$$

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$

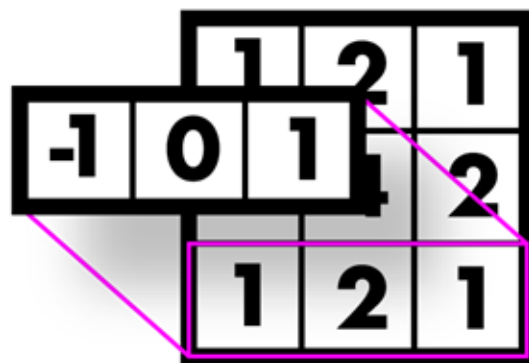


$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



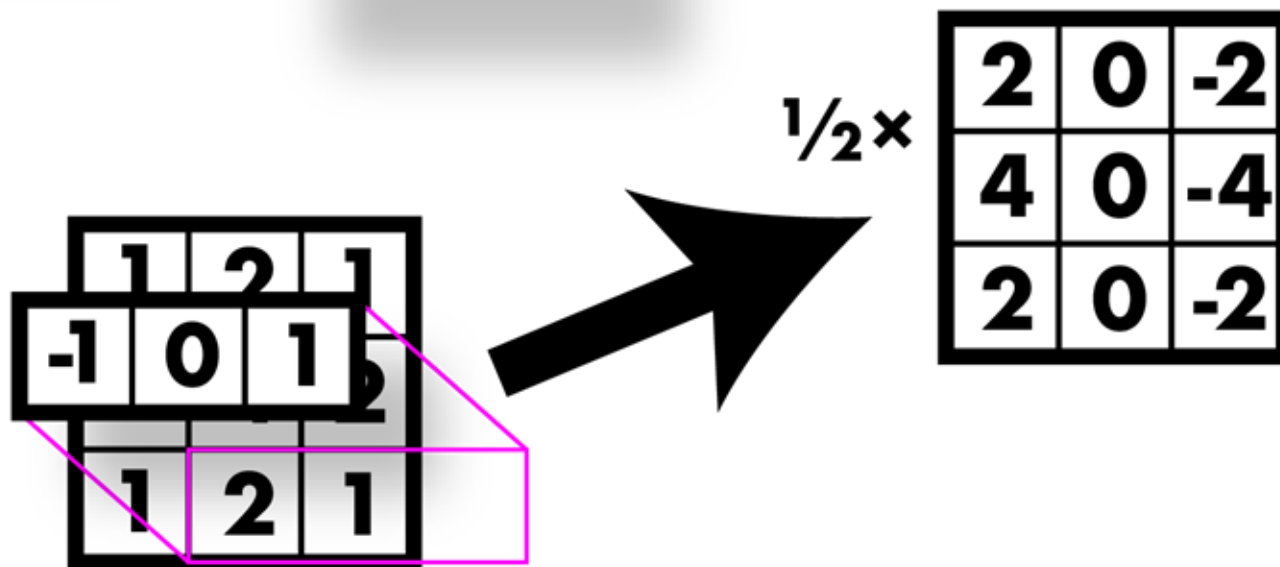
$$\frac{1}{2} \times \begin{bmatrix} 2 & 0 & -2 \\ 4 & 0 & -4 \\ 2 & & \end{bmatrix}$$

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



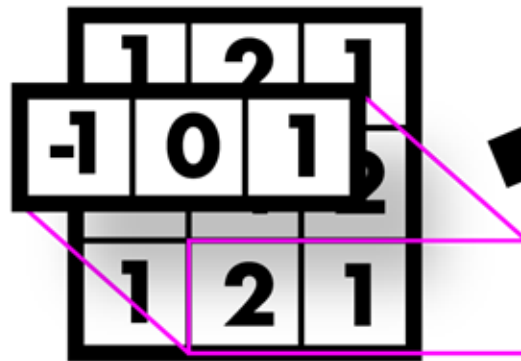
$$\frac{1}{2} \times \begin{bmatrix} 2 & 0 & -2 \\ 4 & 0 & -4 \\ 2 & 0 & \end{bmatrix}$$

$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



Sobel filter → smooth & derivative

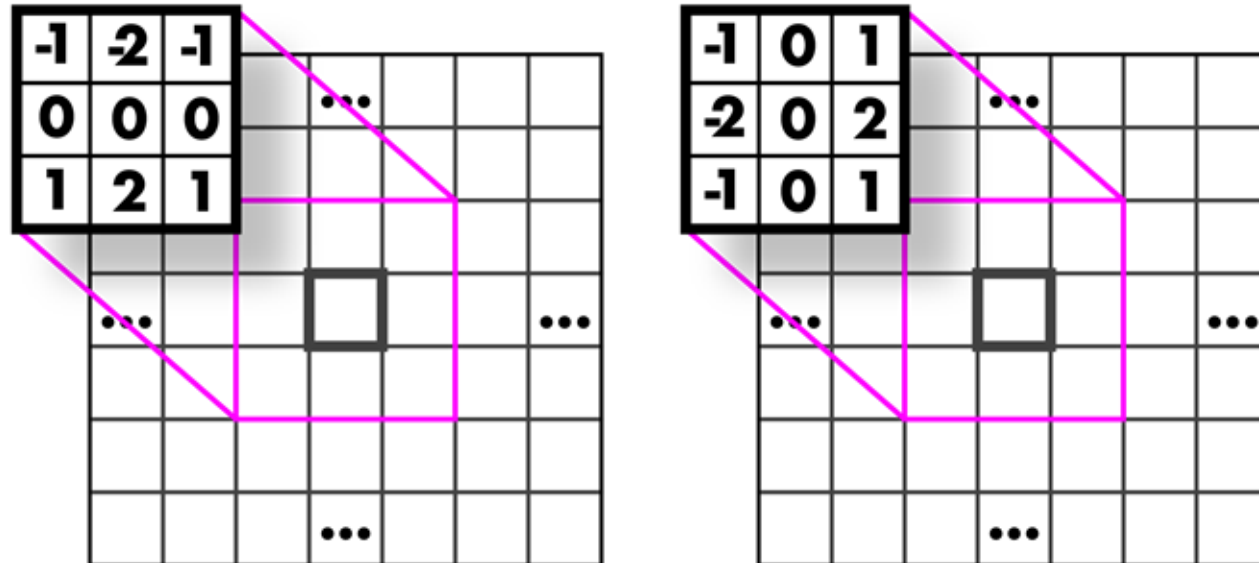
$$\frac{1}{2} \times \left(\begin{bmatrix} -1 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \right)$$



1	0	-1
2	0	-2
1	0	-1

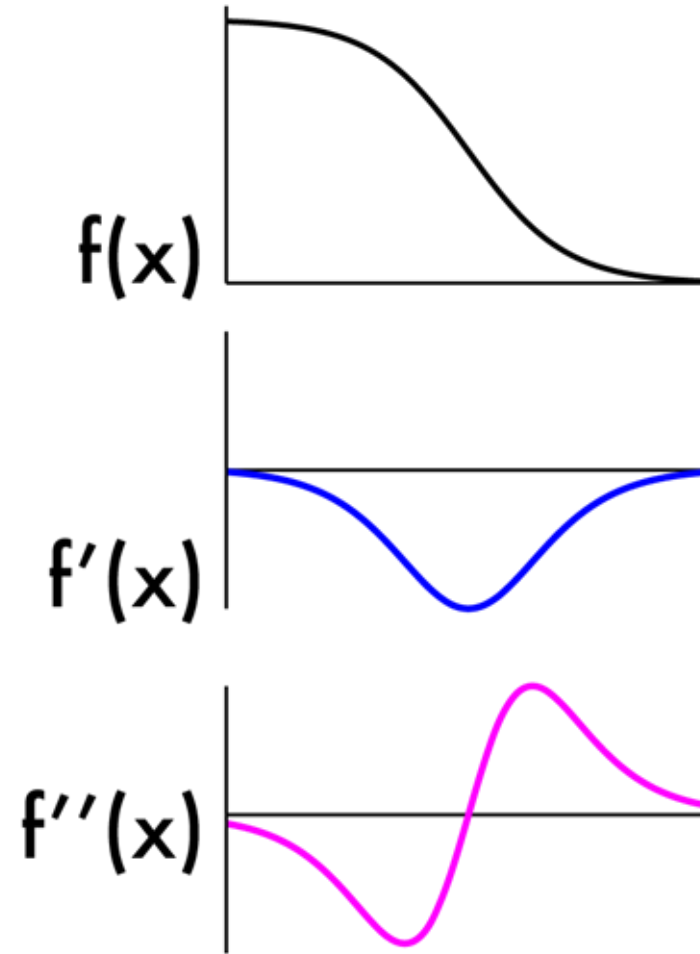
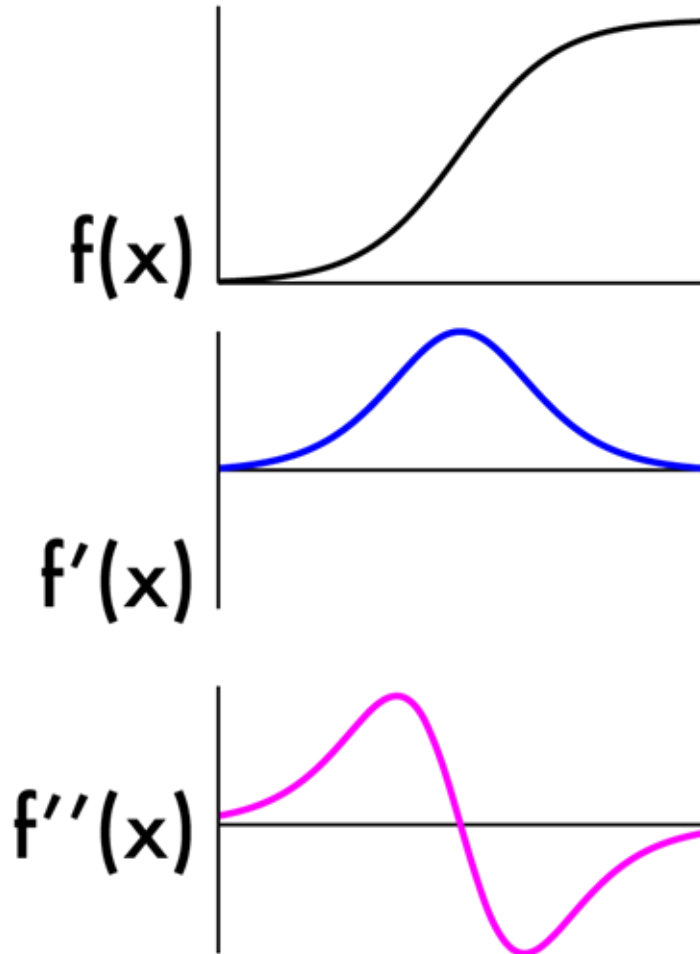
Finding edges

- Could take derivative
 - Find high responses $\rightarrow$ Sobel filters!



Finding edges

- Alternatively, take 2nd derivative $\rightarrow$ crosses zero at extrema



Finding edges

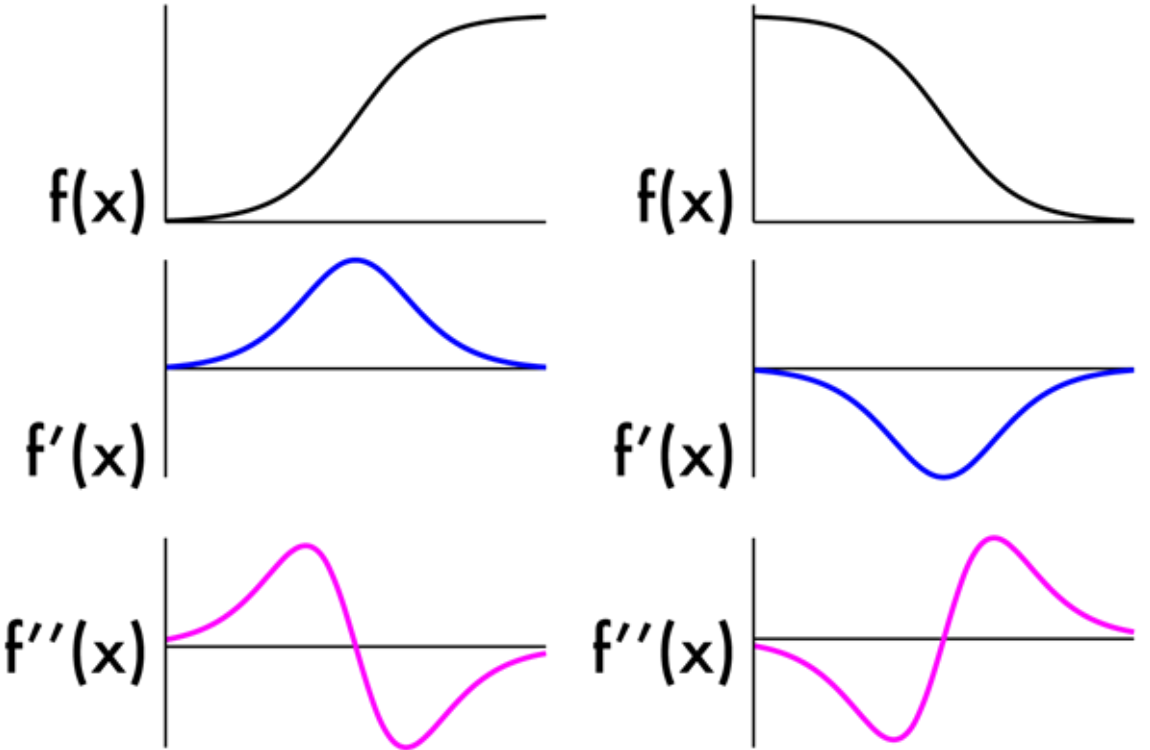
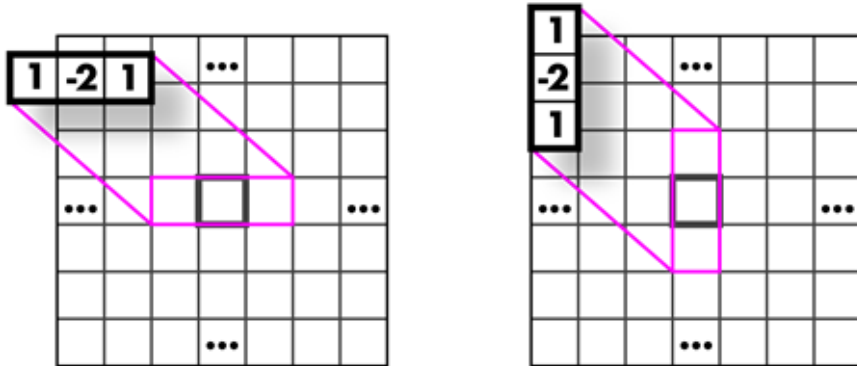
- Crosses zero at extrema

- Recall:

$$f''(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}.$$

- Laplacian: $\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

- Again, have to estimate $f''(x)$:



Laplacians

- Laplacian:

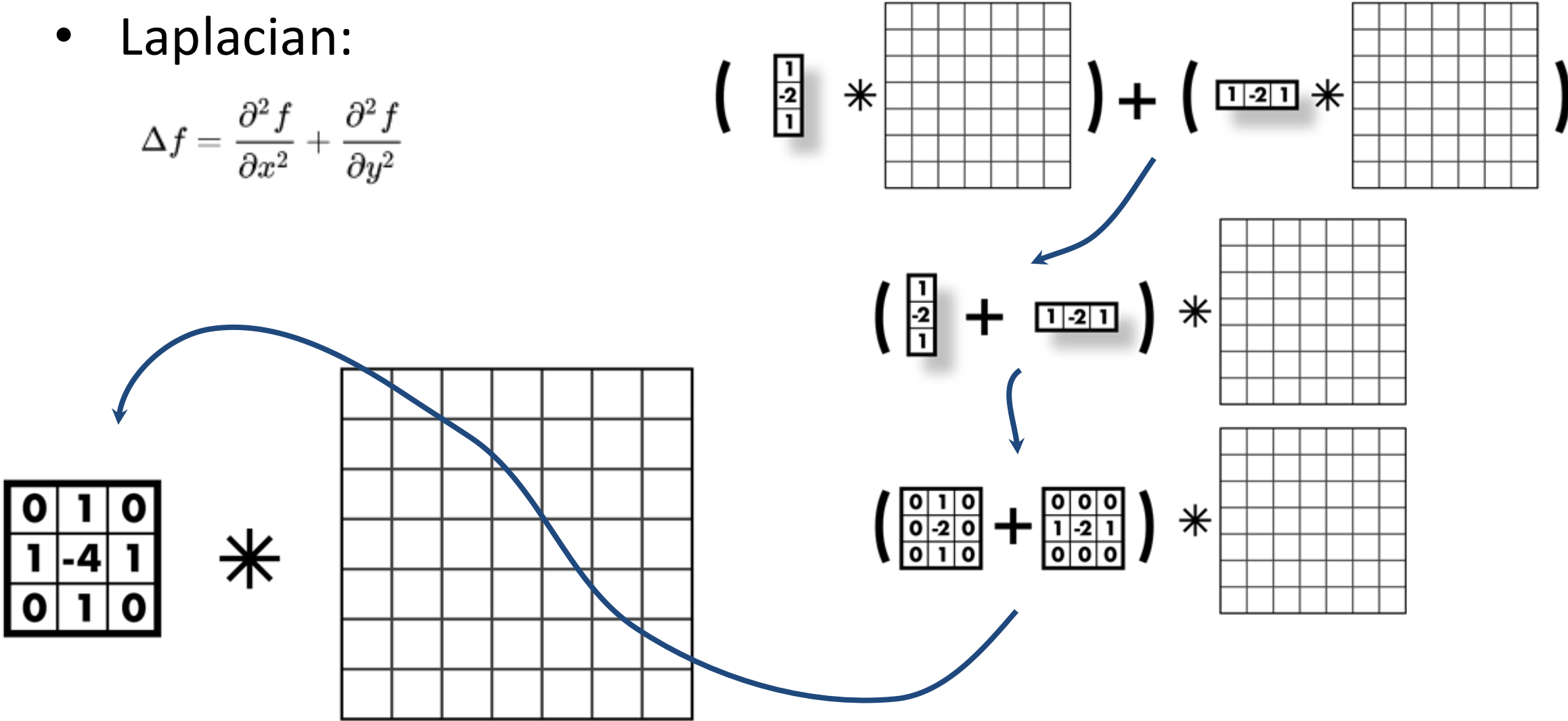
$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

- Measures the divergence of the gradient (1st derivative)
 - Flux of gradient vector field through small area

Laplacians

- Laplacian:

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$



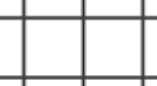
Laplacians

- Laplacian:

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Negative Laplacian, -4 in middle

0	1	0
1	-4	1
0	1	0



Positive Laplacian

“Extended” Laplacian

-1	-1	-1
-1	8	-1
-1	-1	-1

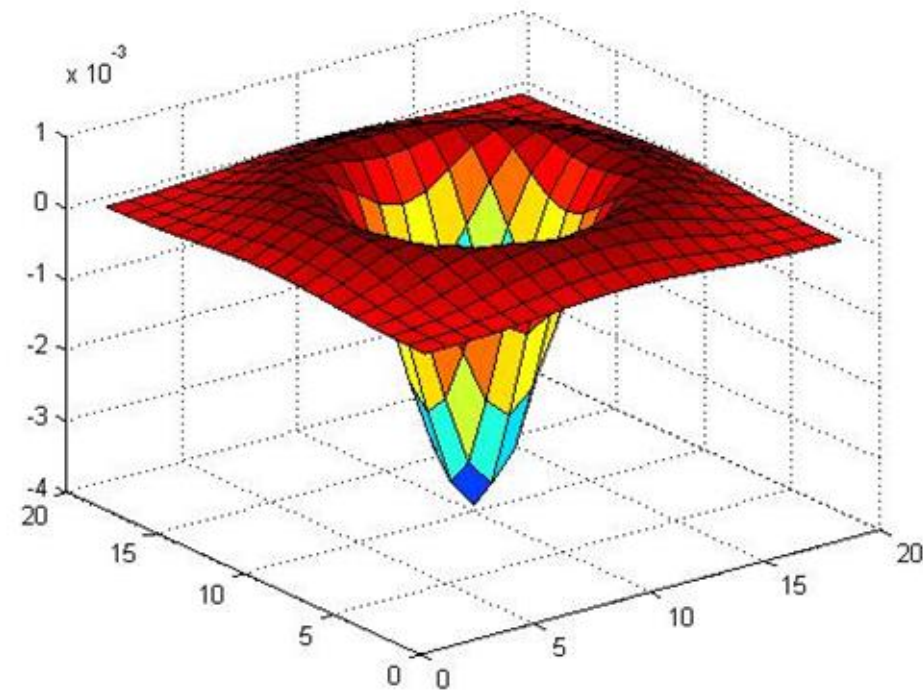
5x5 Laplacian

0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0

Laplacians also sensitive to noise

- Again, use gaussian smoothing
- Can just use one kernel (convolution operation is associative)
- Laplacian of Gaussian, LoG
- Can get good approx. with 5x5 - 9x9 kernels

0	1	1	2	2	2	1	1	0
1	2	4	5	5	5	4	2	1
1	4	5	3	0	3	5	4	1
2	5	3	-12	-24	-12	3	5	2
2	5	0	-24	-40	-24	0	5	2
2	5	3	-12	-24	-12	3	5	2
1	4	5	3	0	3	5	4	1
1	2	4	5	5	5	4	2	1
0	1	1	2	2	2	1	1	0



Another edge detector

- Image is a function:
 - Has high frequency and low frequency components
- Edges are **high frequency changes**
- Maybe we want to find **edges of a specific size** (i.e. specific frequency)

Difference of Gaussian (DoG)

- Gaussian is a **low pass filter**
 - Strongly reduce components with frequency $f < \sigma$

- *Low frequency* components

$$g * I$$

- *High frequency* components

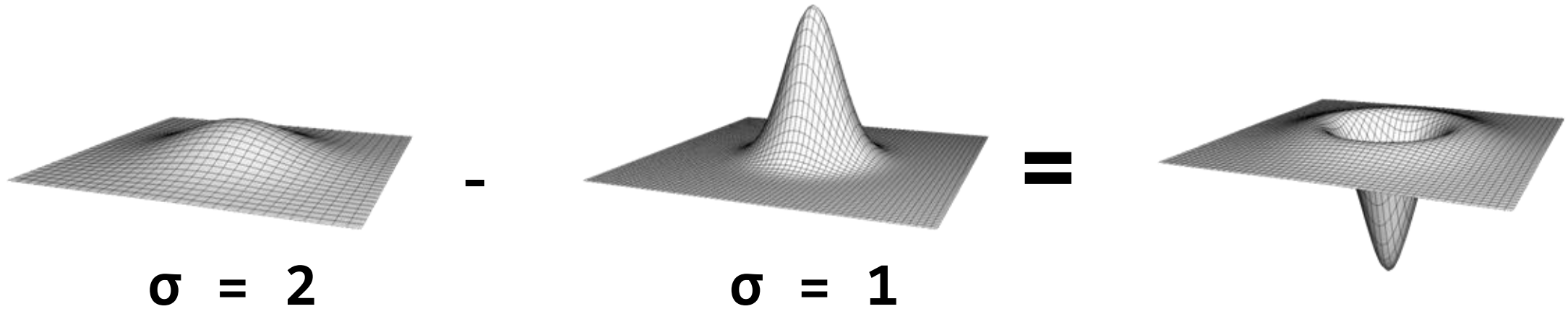
$$I - g * I$$

- Components *between frequencies*

$$\begin{aligned} g(\sigma_1) * I - g(\sigma_2) * I \\ (g(\sigma_1) - g(\sigma_2)) * I \end{aligned}$$

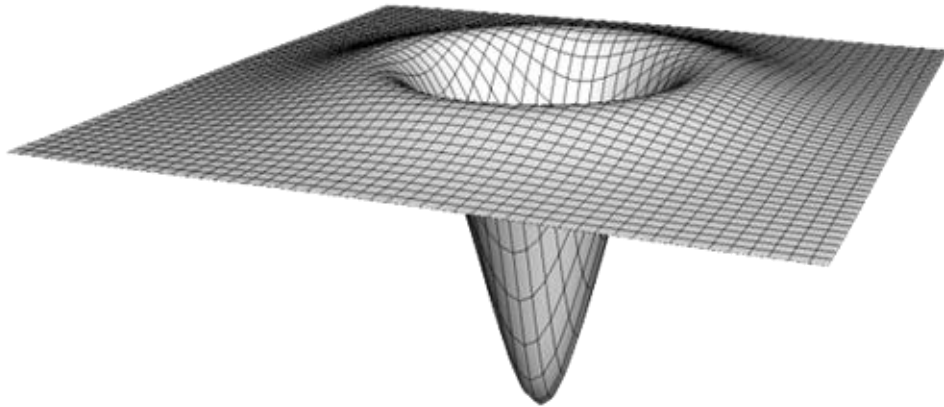
Difference of Gaussian (DoG)

$$g(\sigma_1) * I - g(\sigma_2) * I = (g(\sigma_1) - g(\sigma_2)) * I$$

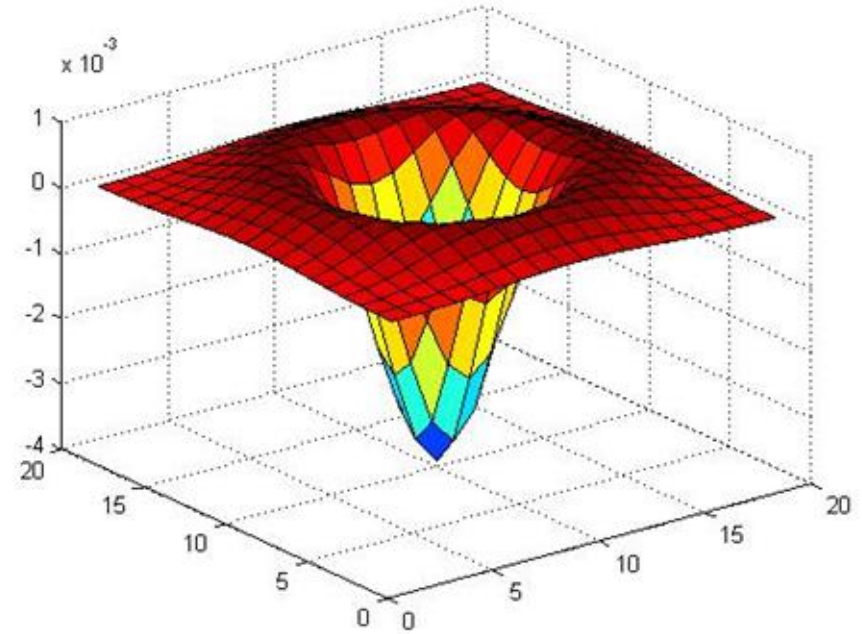


Difference of Gaussian (DoG)

- $g(\sigma_1) * I - g(\sigma_2) * I = (g(\sigma_1) - g(\sigma_2)) * I$
- This looks a lot like a LoG
 - (not actually the same but similar)



DoG



LoG

DoG (1 - 0)



DoG (2 - 1)



DoG (3 - 2)

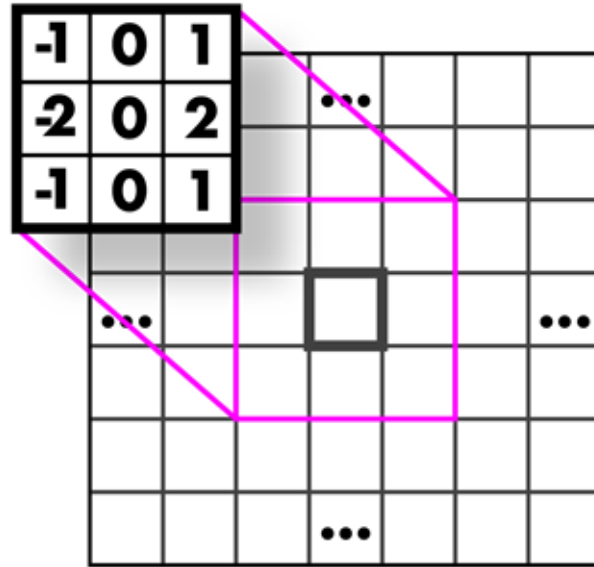
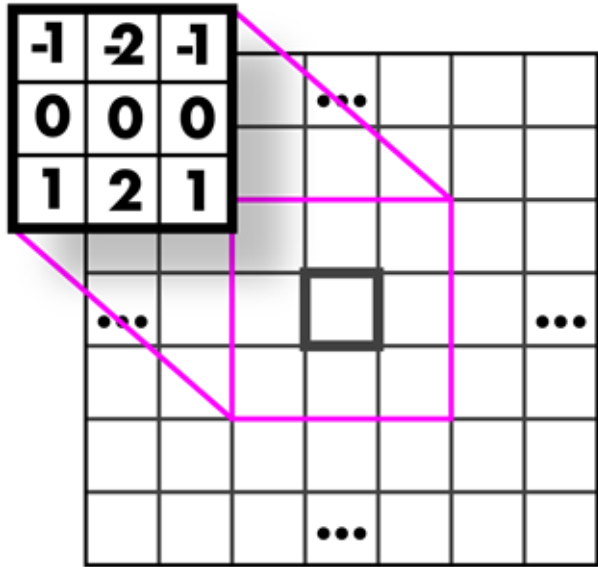


DoG (4 - 3)

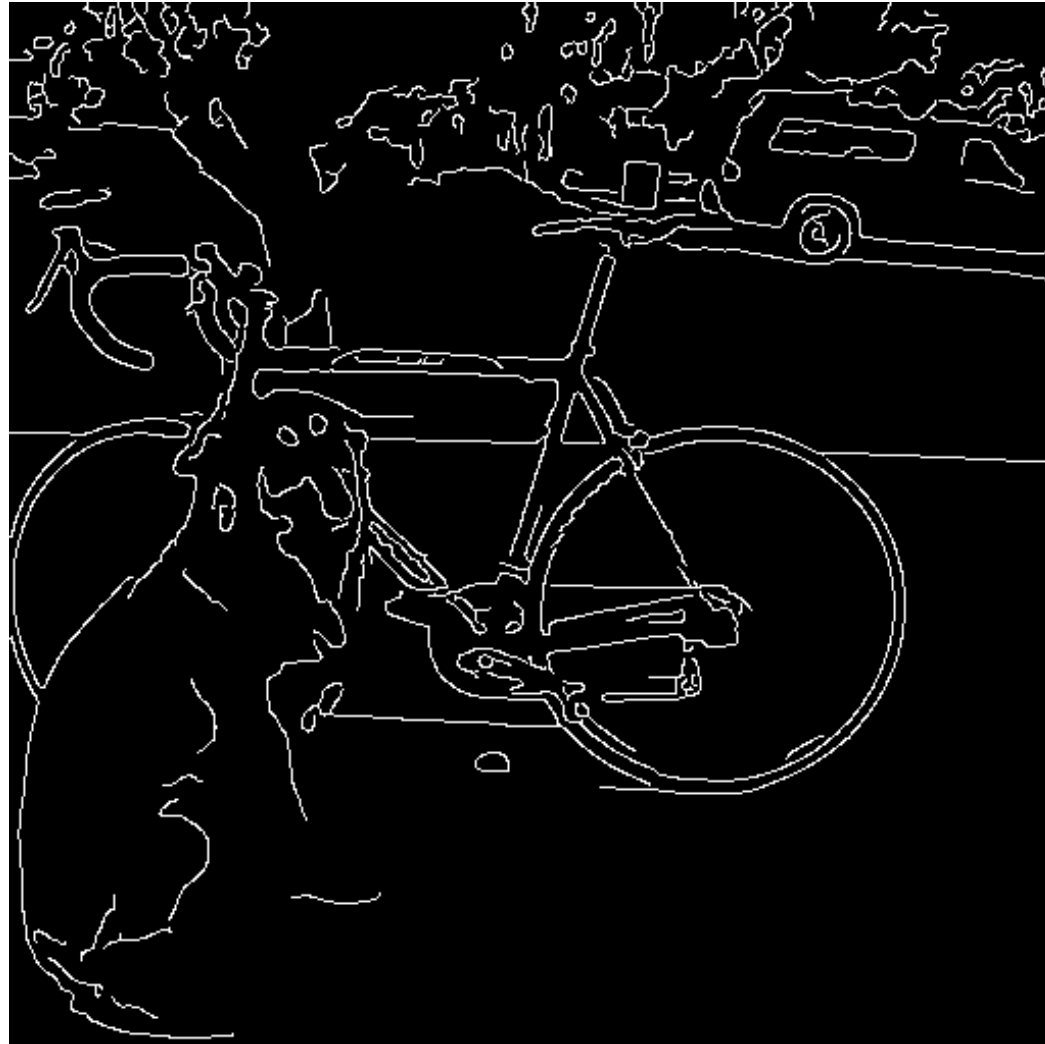


Another approach: gradient magnitude

- Don't need 2nd derivatives
- Just use magnitude of gradient
- Are we done?



What we really want: lines



Canny Edge Detection

Algorithm:

- Smooth image (only want “real” edges, not noise)
- Calculate gradient direction and magnitude
- Non-maximum suppression perpendicular to edge
- Threshold into strong, weak, no edge
- Connect together components

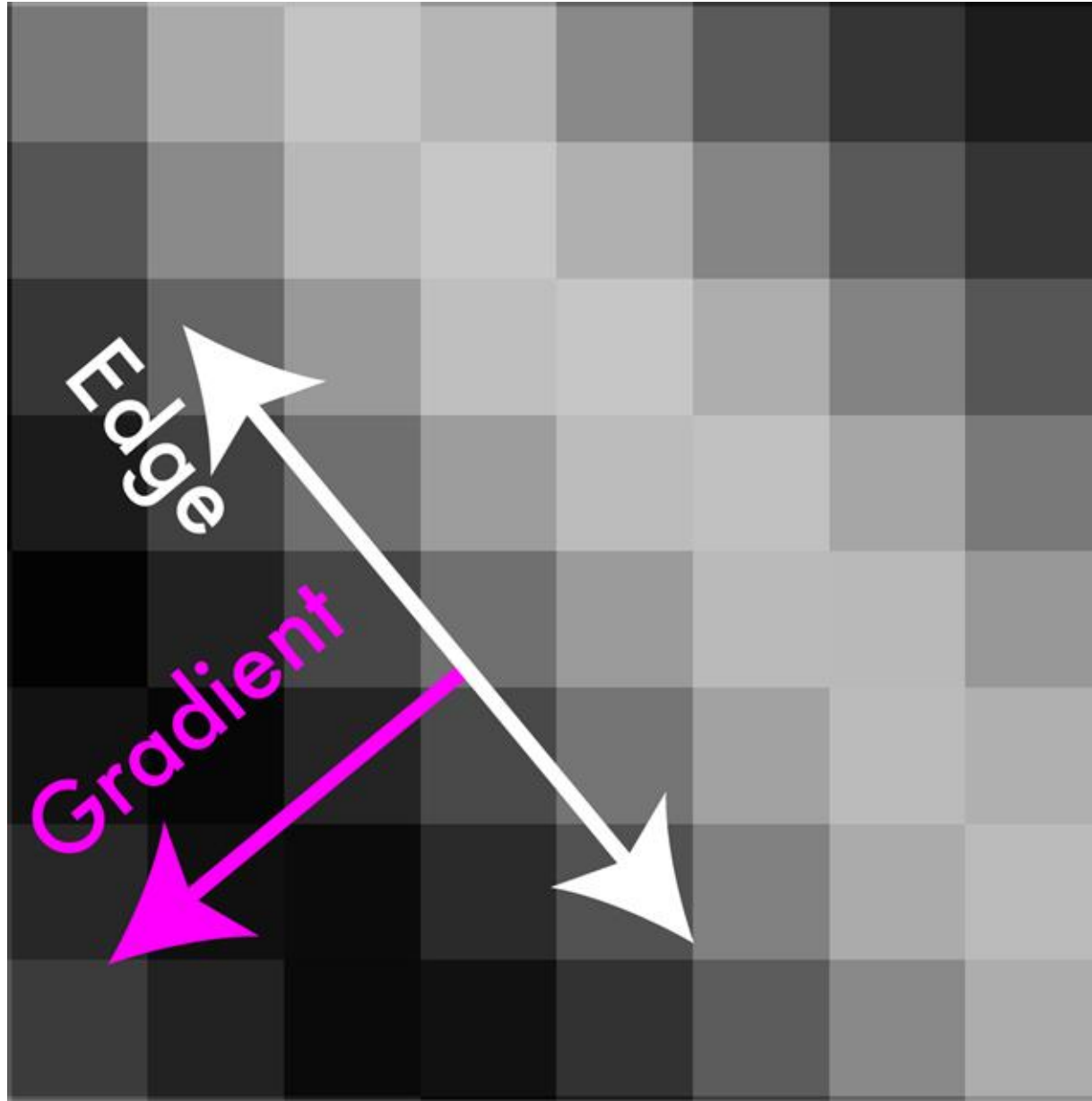


Non-maximum suppression

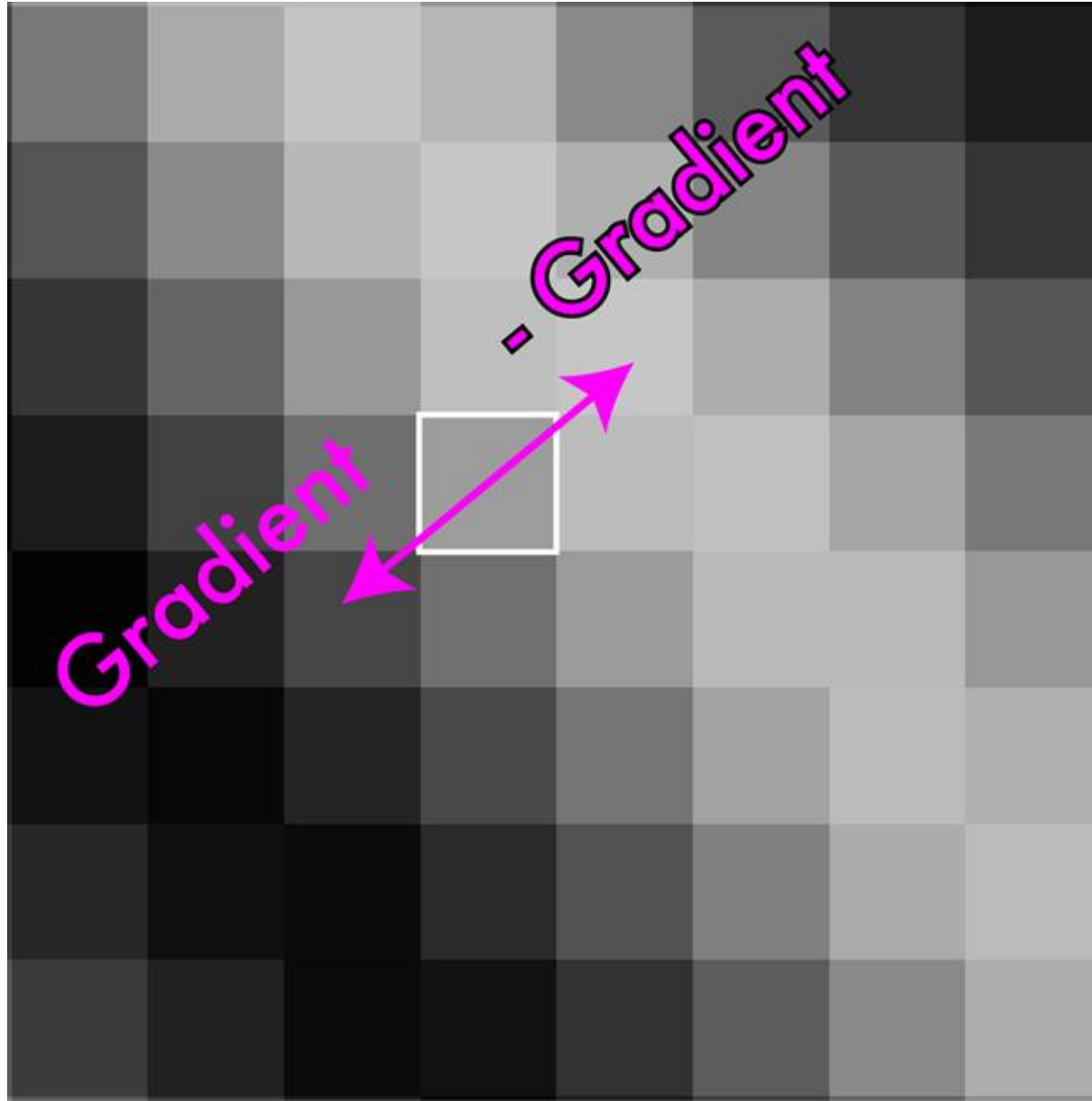
- Want **single pixel edges**, not thick blurry lines
- Need to **check nearby pixels**
- See if response is **highest**



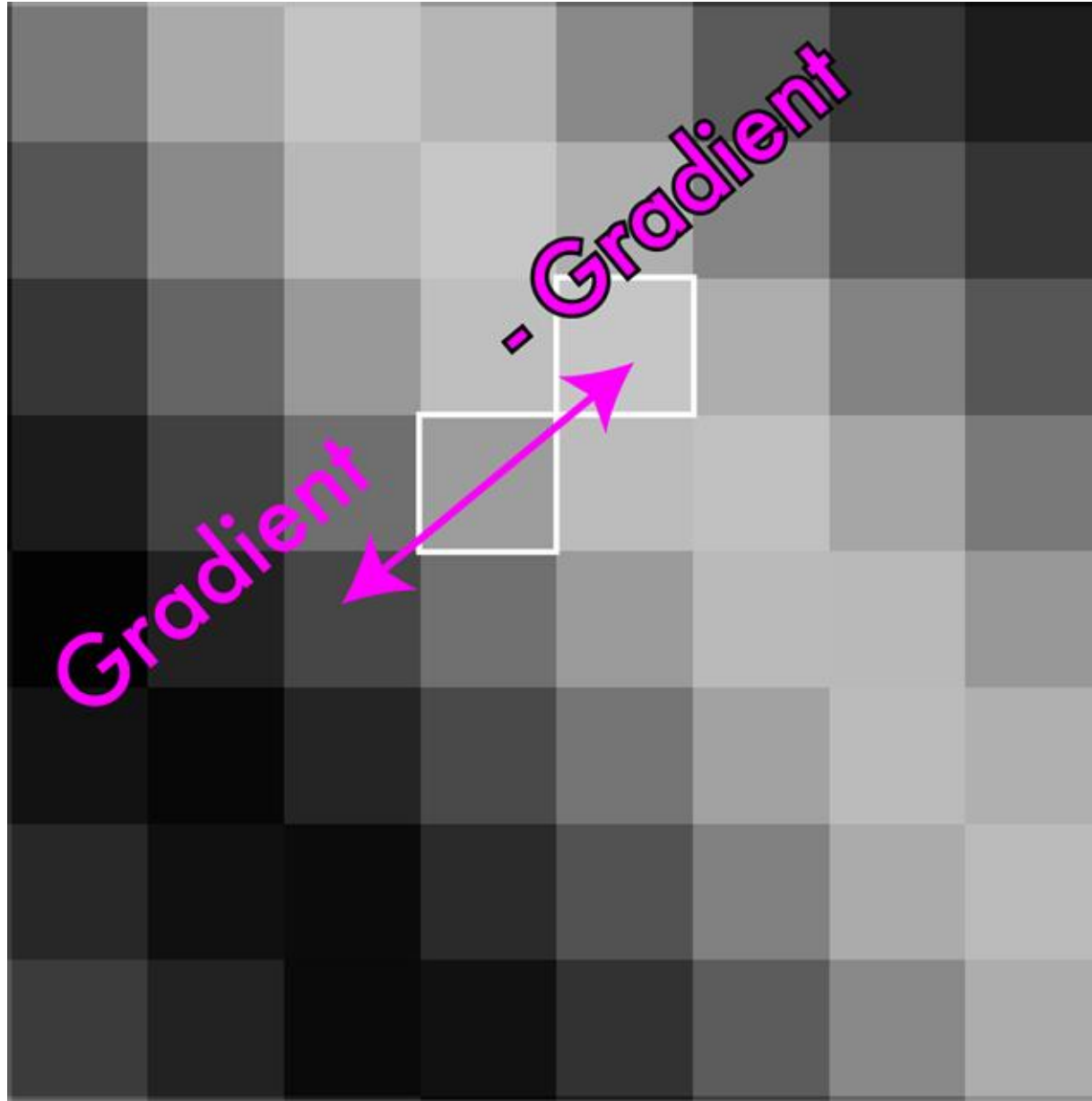
Non-maximum suppression



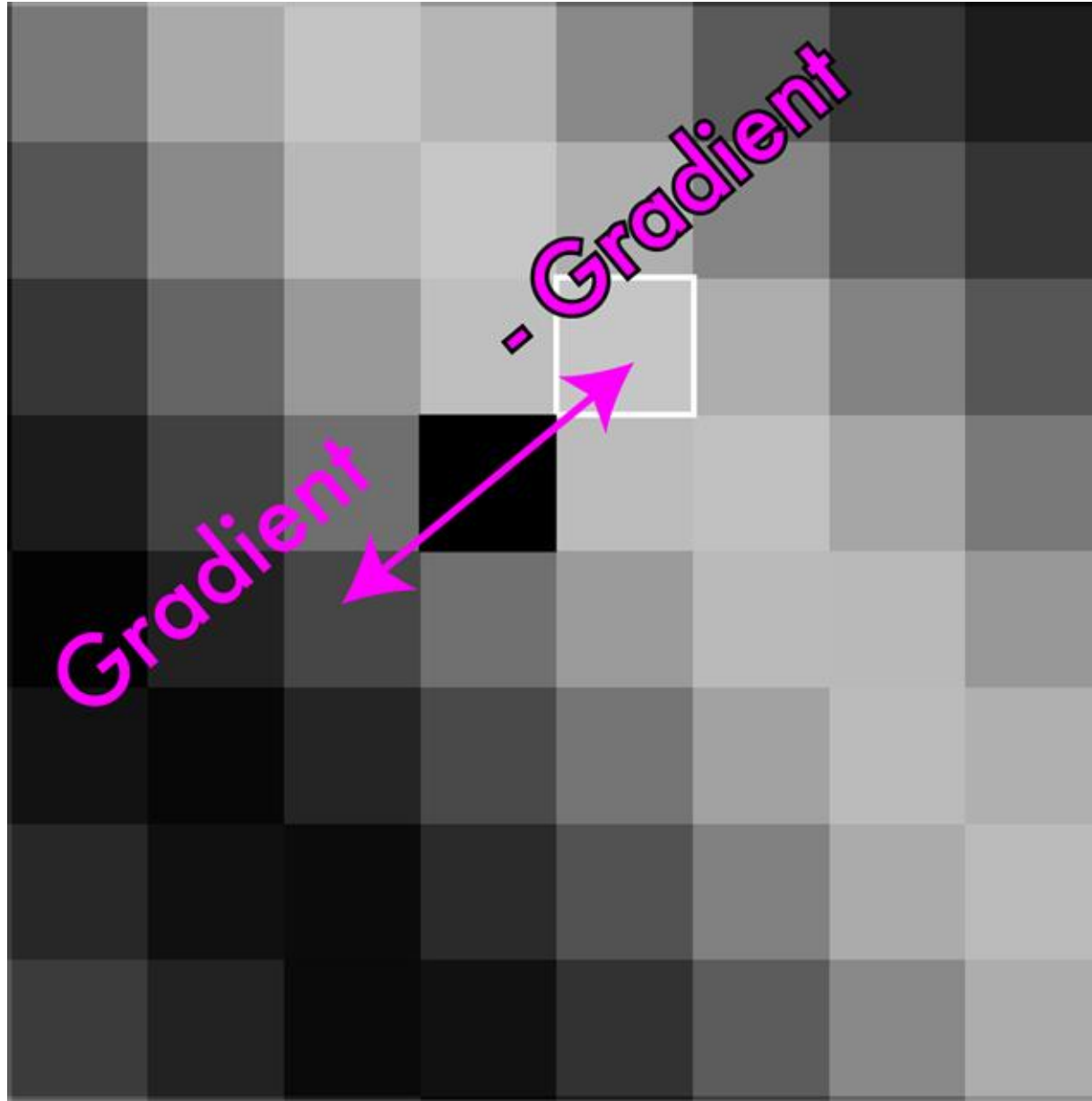
Non-maximum suppression



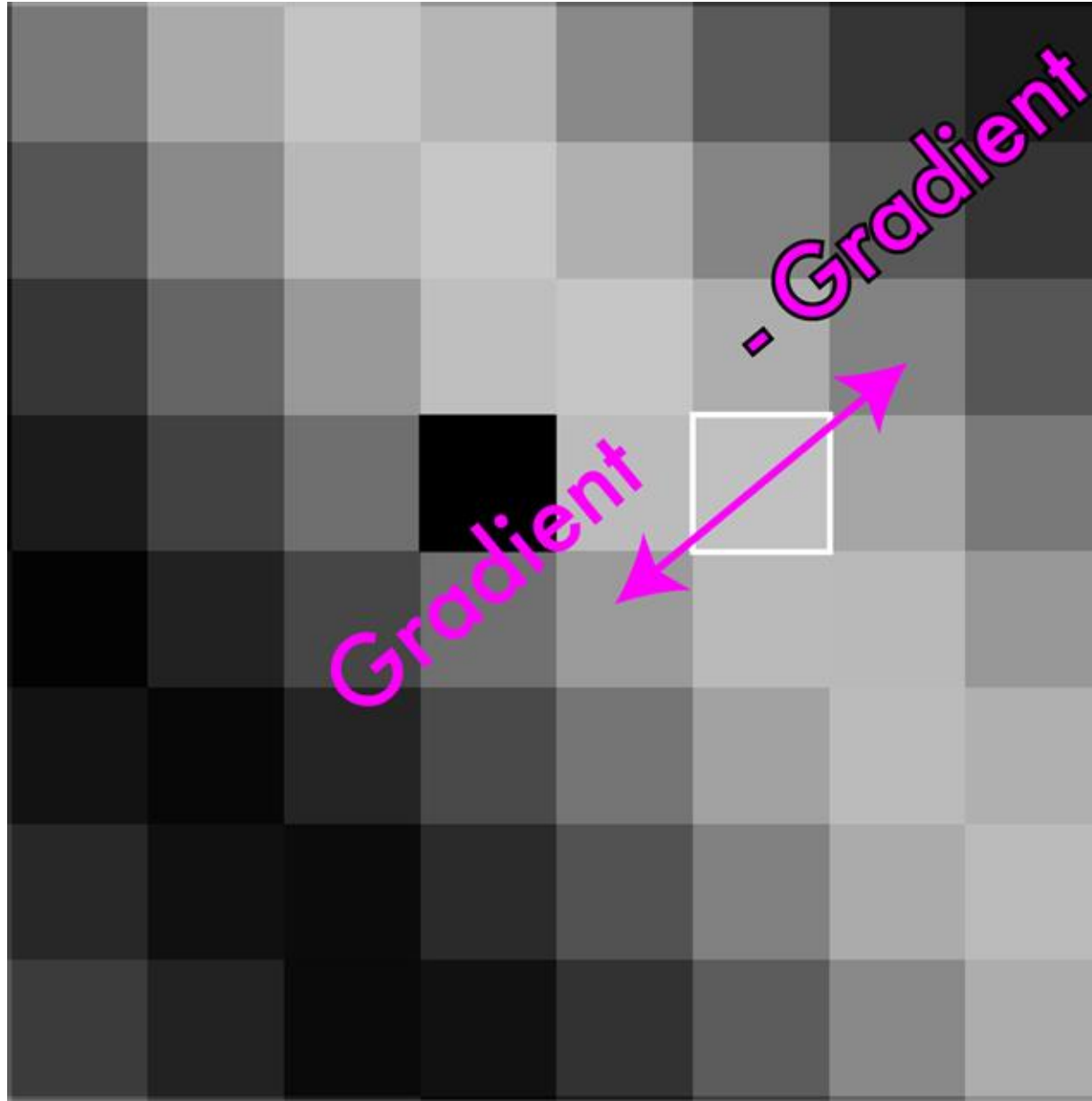
Non-maximum suppression



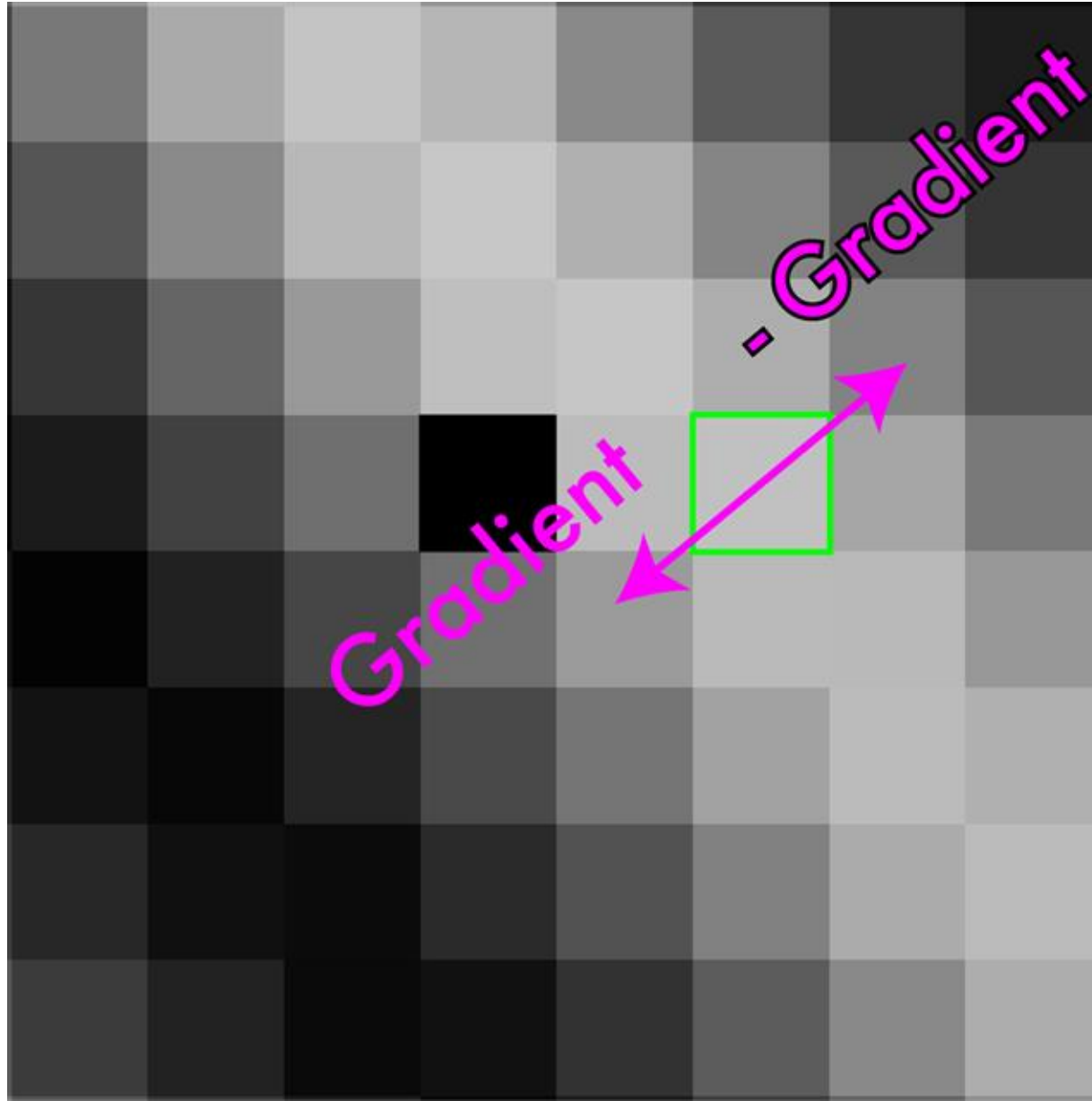
Non-maximum suppression



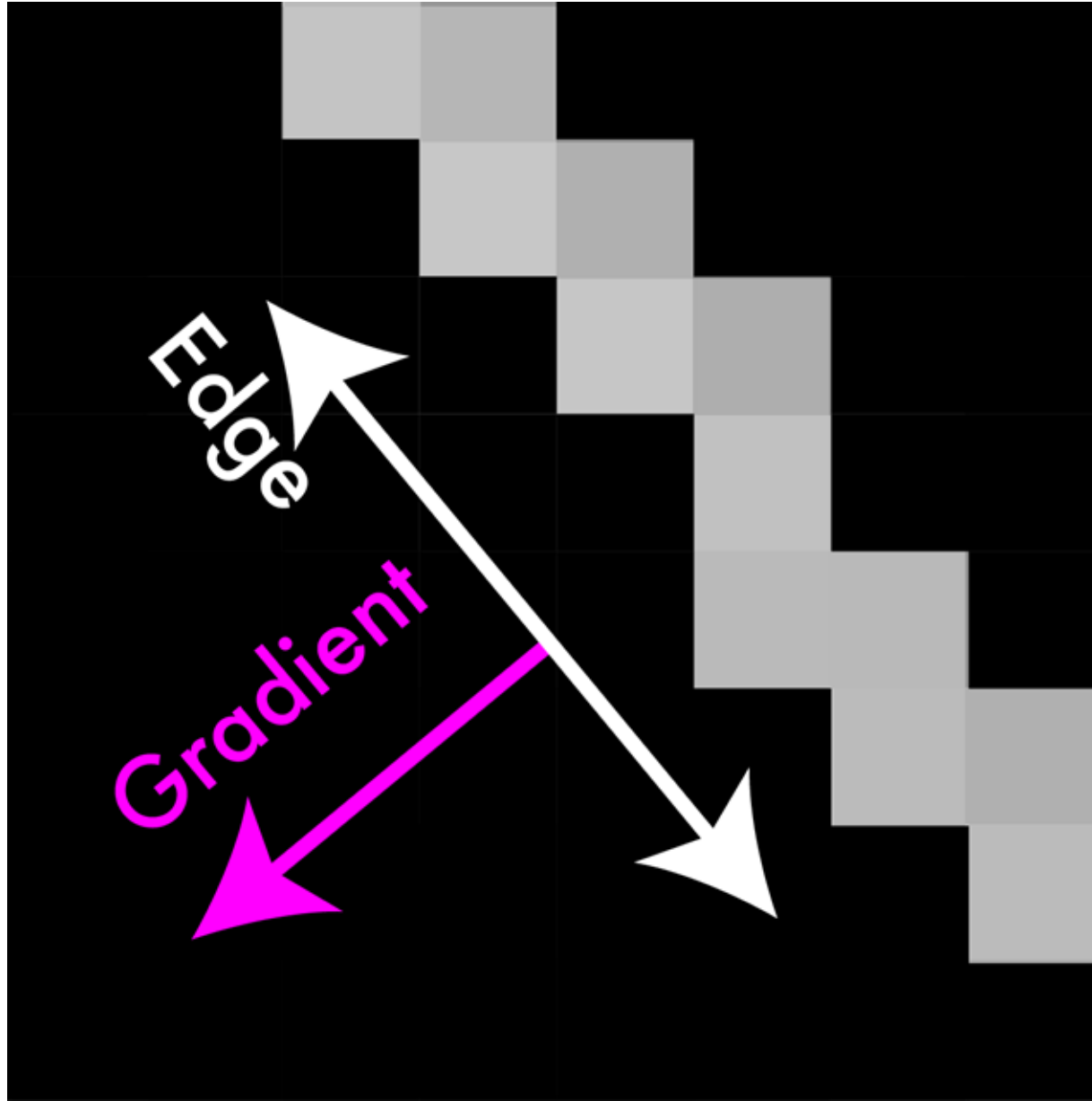
Non-maximum suppression



Non-maximum suppression



Non-maximum suppression



Non-maximum suppression



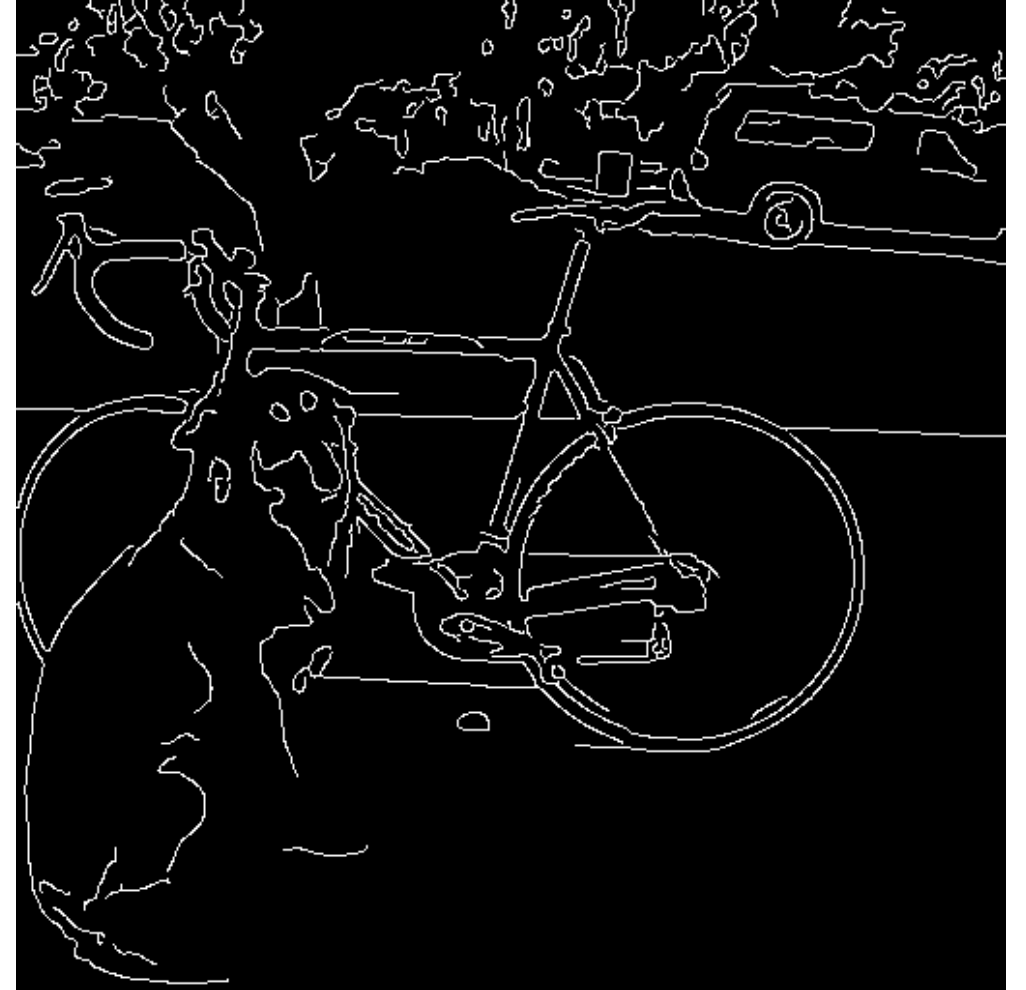
Threshold edges

- Still need an additional step
- Only want strong edges
- 2 thresholds, 3 cases
 - Hysteresis threshold
 - $R > T$: strong edge
 - $R < T$ but $R > t$: weak edge
 - $R < t$: no edge
- Why two thresholds?



Connect edges

- Strong edges are edges
- Weak edges are edges **iff** they connect to strong edges
- Look in some neighborhood (usually 8 closest)



Canny Edge Detection

Algorithm:

- Smooth image (only want “real” edges, not noise)
- Calculate gradient direction and magnitude
- Non-maximum suppression perpendicular to edge
- Threshold into strong, weak, no edge
- Connect together components

Parameters: Sigma, thresholds

Image processing pipeline → Old-school CV is all about pipelines

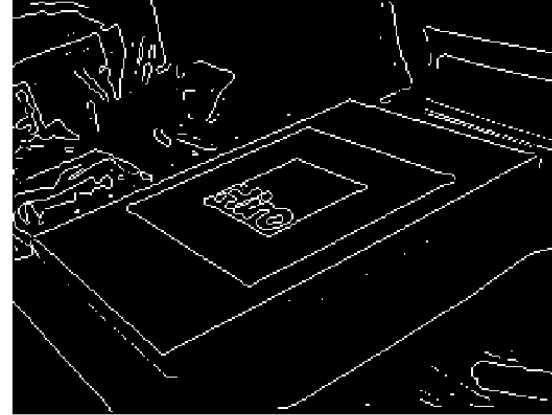
Canny Edge Detection



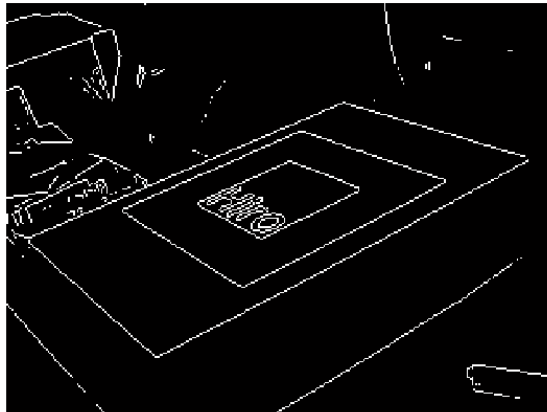
Comparison of Edge Detectors



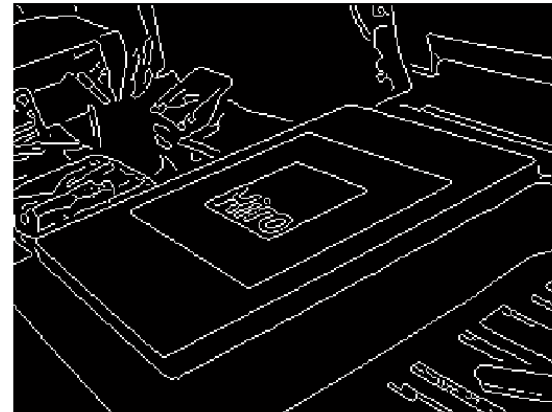
Original image



LoG



Sobel



Canny

detection thresholds automatically chosen

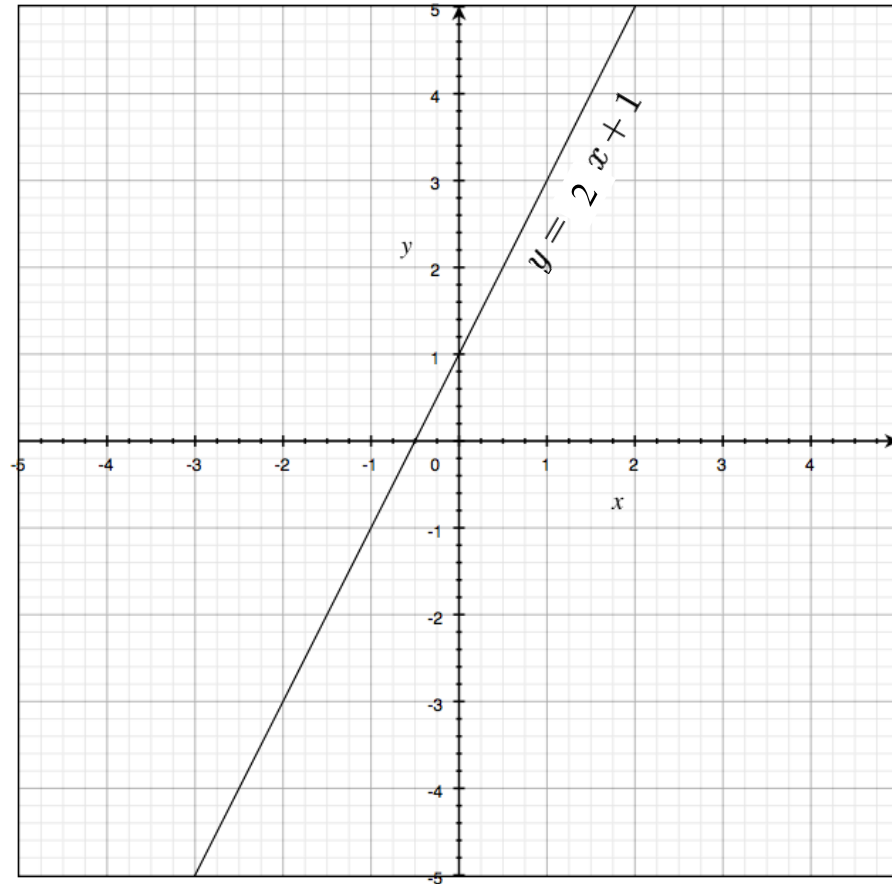
Edges → Lines

Line parameterizations

Slope intercept form

$$y = mx + b$$

 **slope**  **y-intercept**



Hough transform

- Generic framework for detecting a parametric model
- Edges don't have to be connected
- Lines can be occluded
- Key idea: edges **vote** for the possible models

Image and parameter space

variables

$$y = mx + b$$

parameters

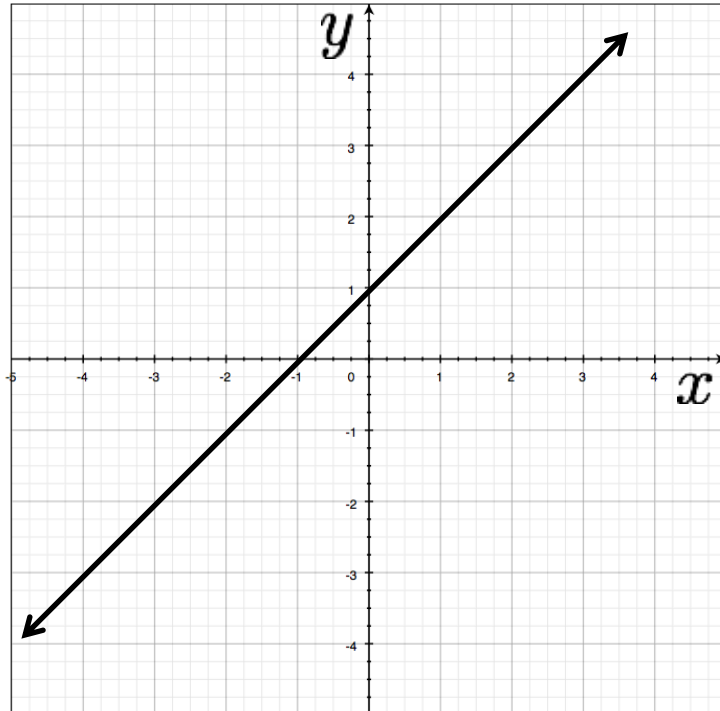


Image space

Image and parameter space

variables

$$y = mx + b$$

parameters

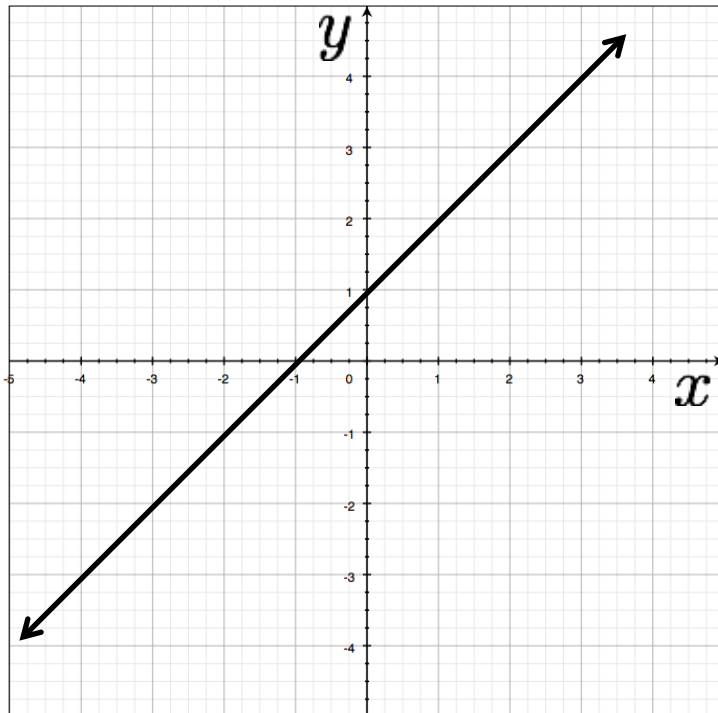
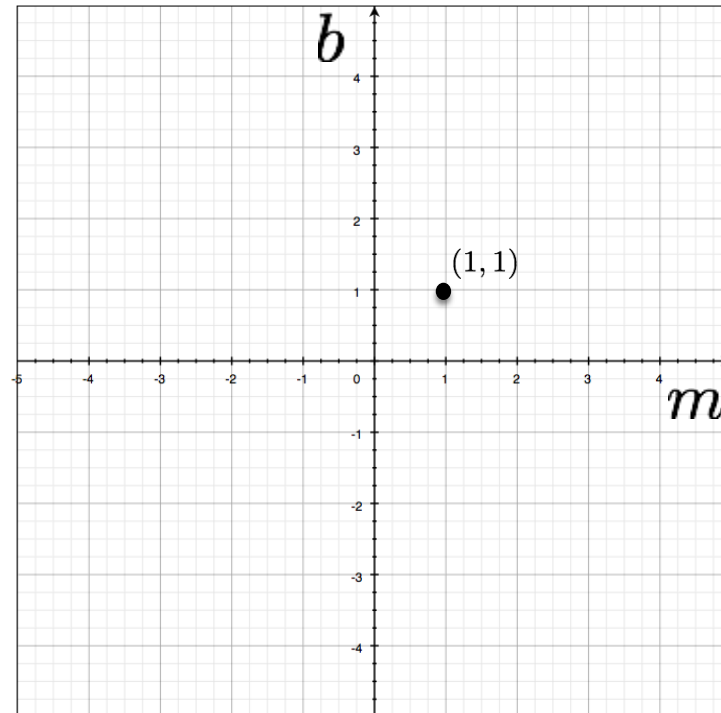


Image space

variables

$$y - mx = b$$

parameters



Parameter space

a line
becomes a
point

Image and parameter space

variables

$$y = mx + b$$

parameters

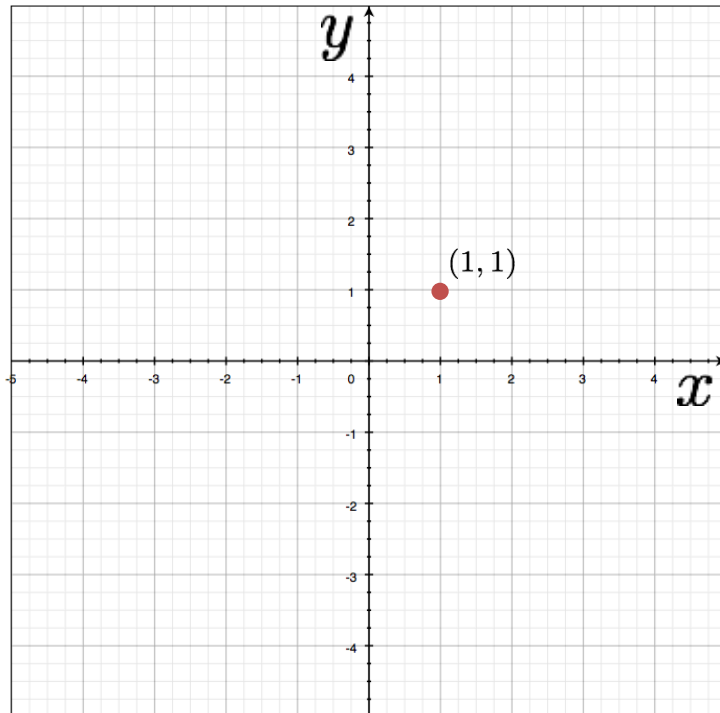


Image space

What would a point in image space become in parameter space?

Image and parameter space

variables

$$y = mx + b$$

parameters

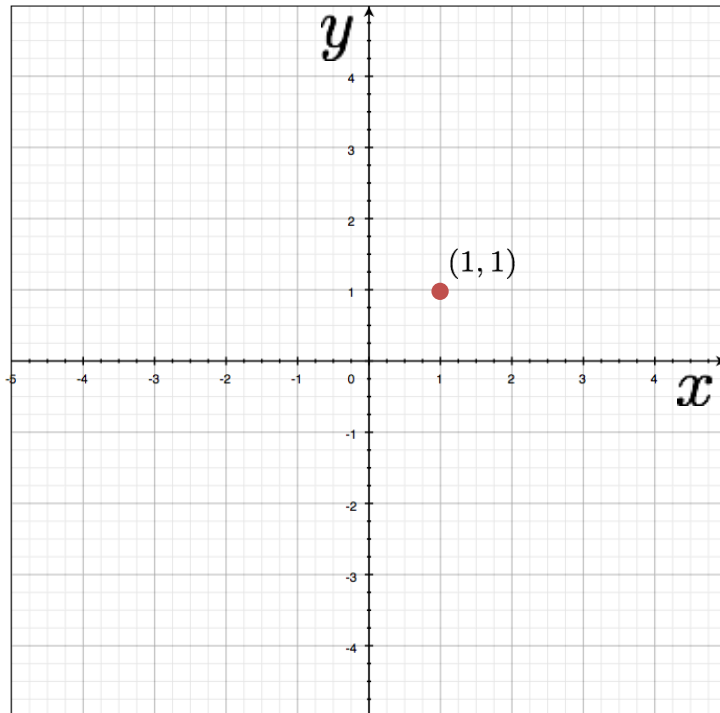


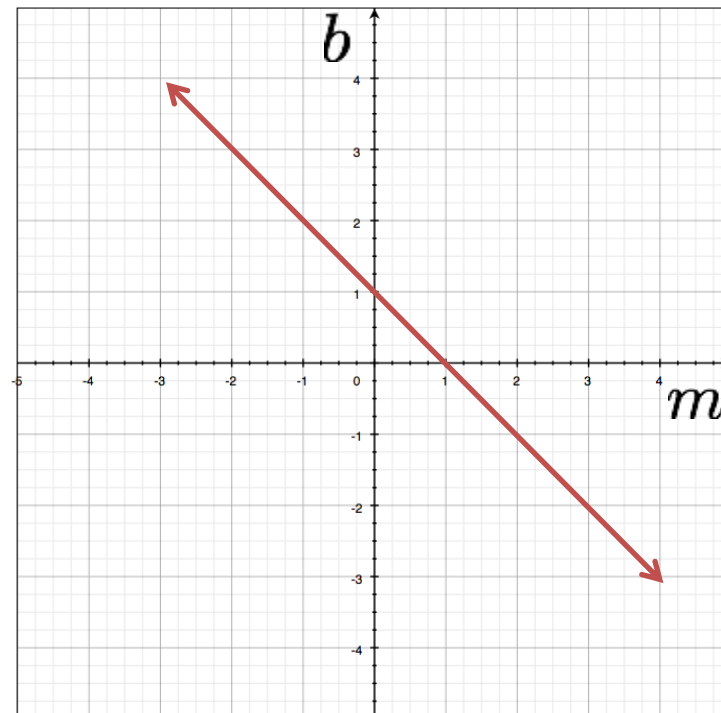
Image space

a point
becomes a
line

variables

$$y - mx = b$$

parameters



Parameter space

Image and parameter space

variables

$$y = mx + b$$

parameters

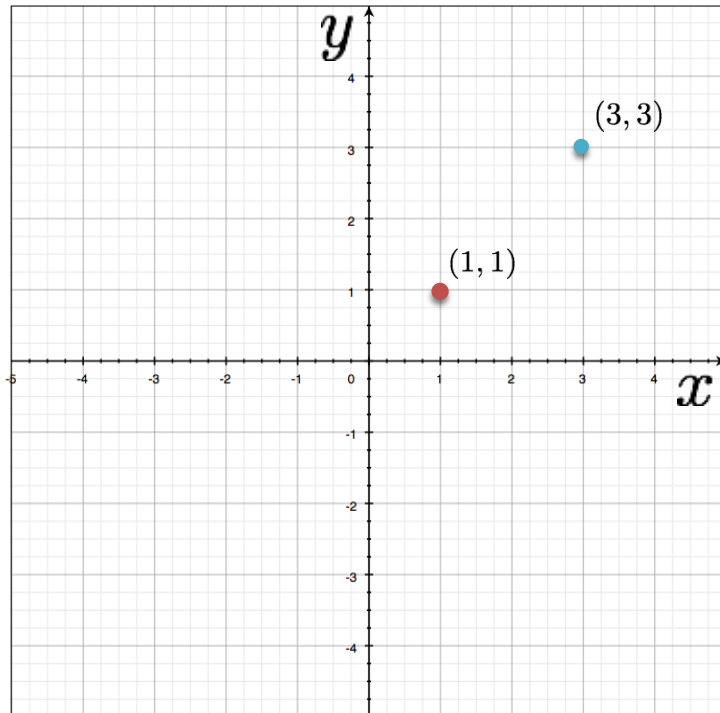
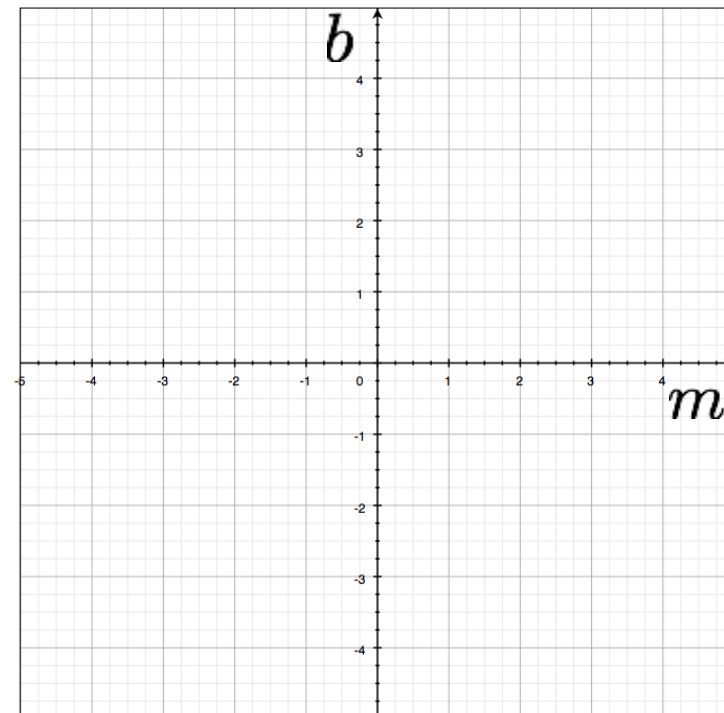


Image space

variables

$$y - mx = b$$

parameters



Parameter space

two points
become
?

Image and parameter space

variables

$$y = mx + b$$

parameters

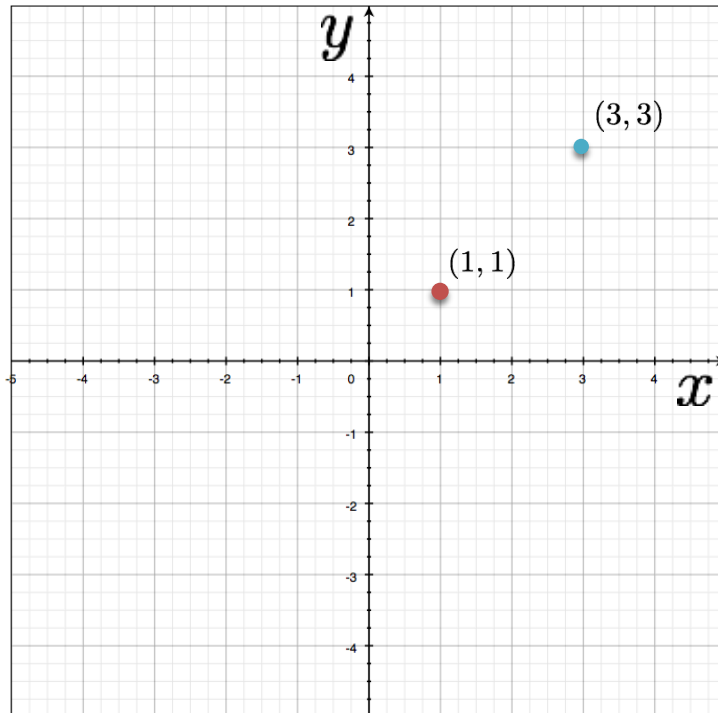


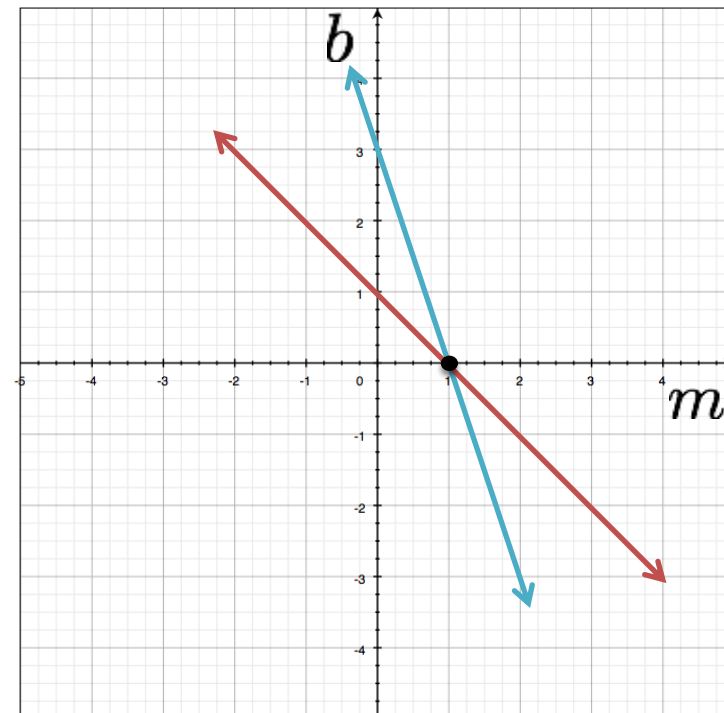
Image space

two points
become
?

variables

$$y - mx = b$$

parameters



Parameter space

Image and parameter space

variables

$$y = mx + b$$

parameters

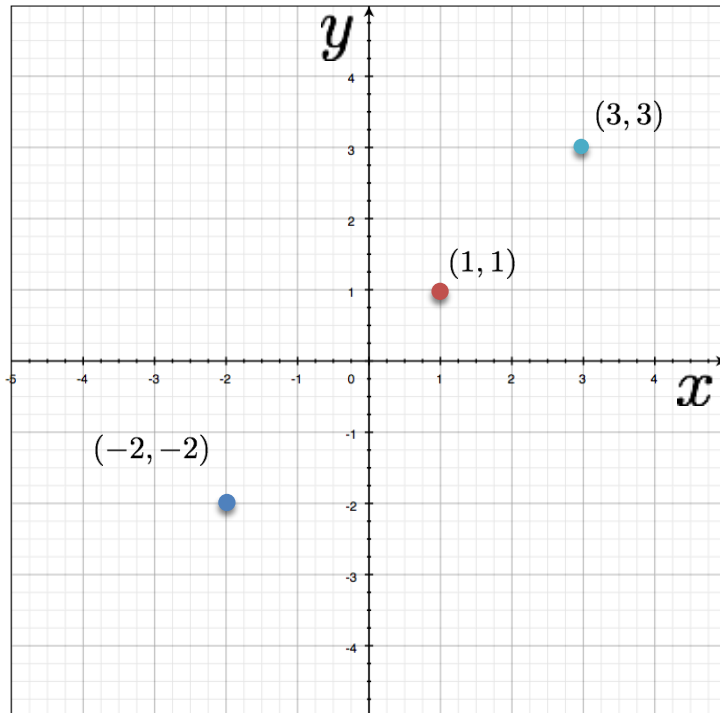


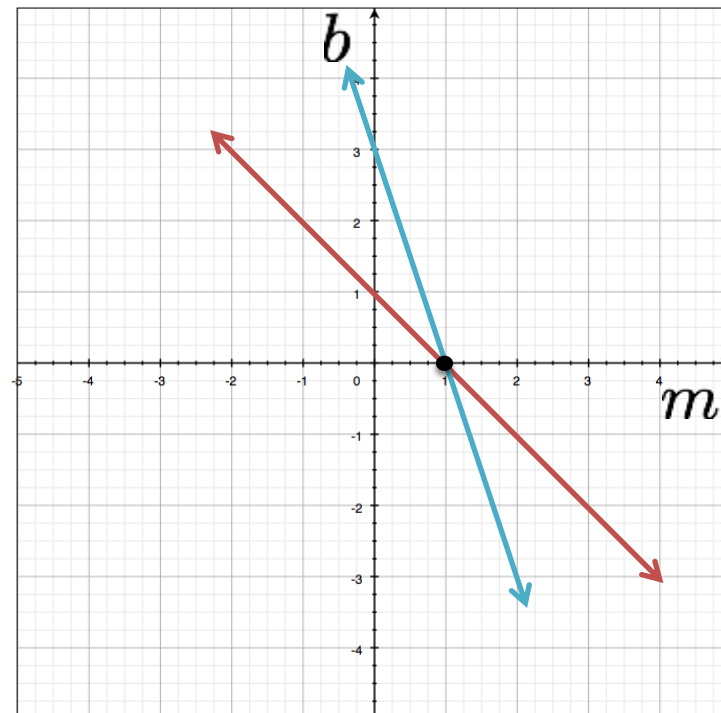
Image space

three points
become
?

variables

$$y - mx = b$$

parameters



Parameter space

Image and parameter space

variables

$$y = mx + b$$

parameters

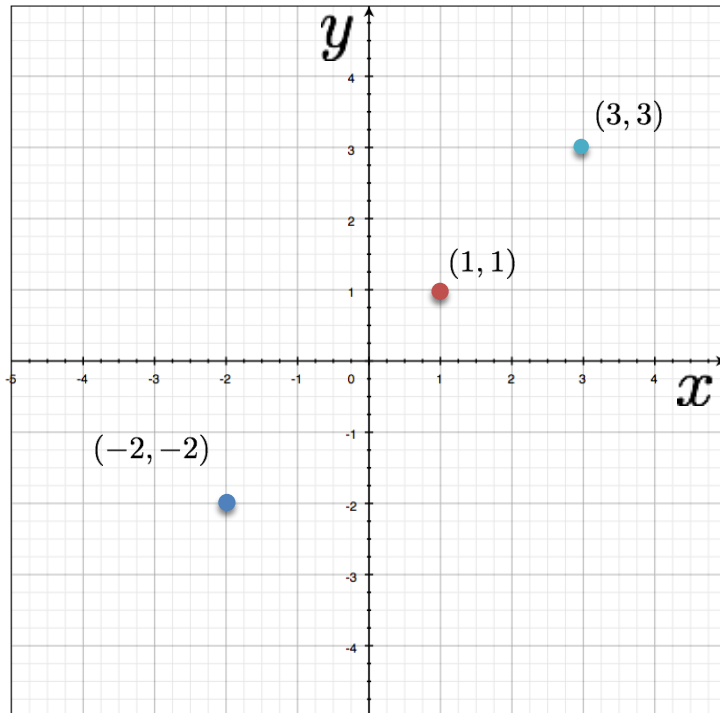


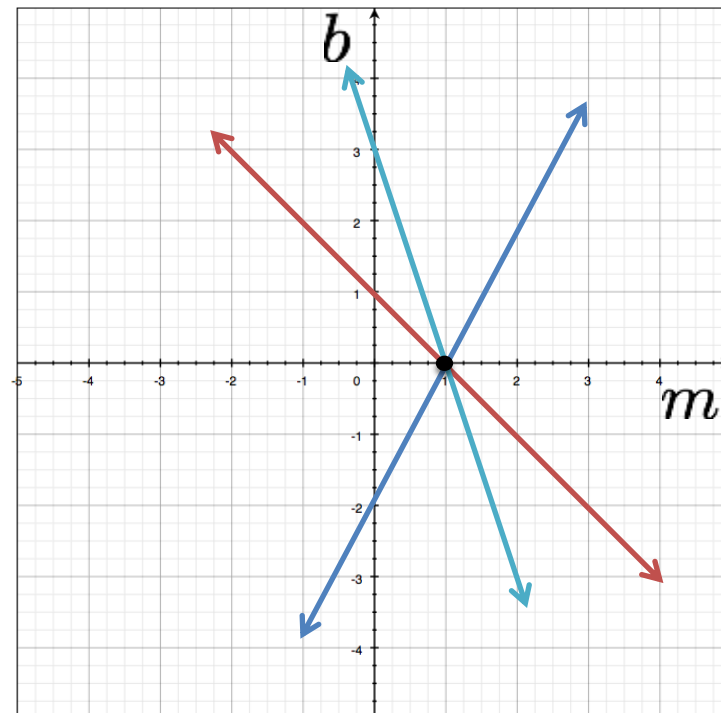
Image space

three points
become
?

variables

$$y - mx = b$$

parameters



Parameter space

Image and parameter space

variables

$$y = mx + b$$

parameters

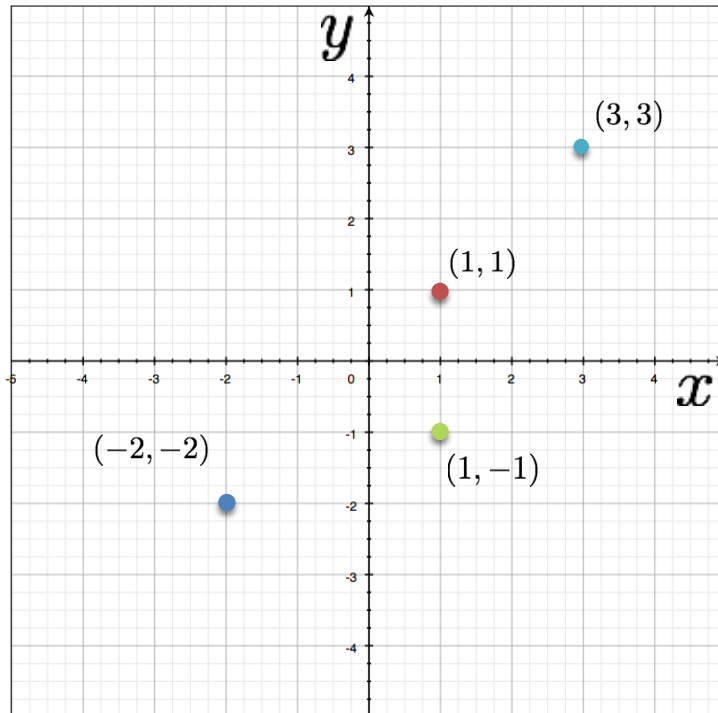


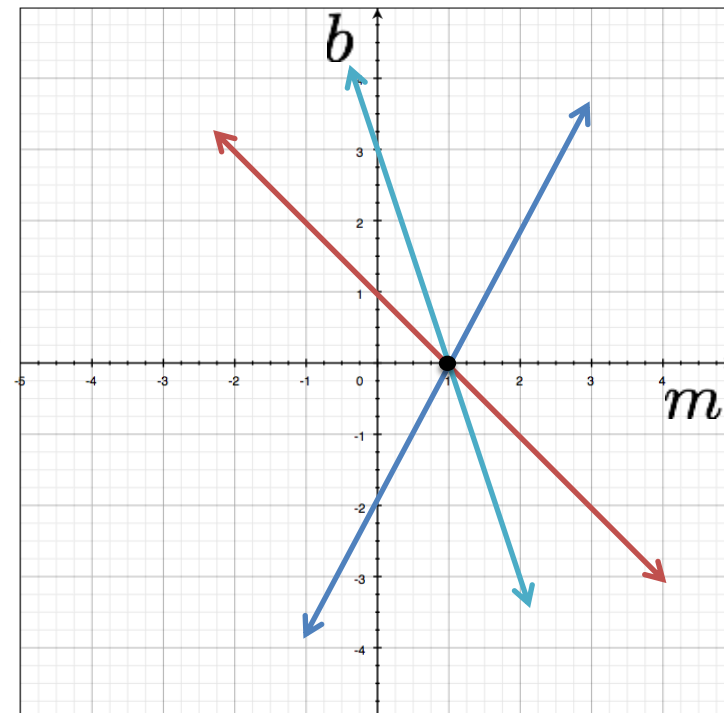
Image space

four points
become
?

variables

$$y - mx = b$$

parameters



Parameter space

Image and parameter space

variables

$$y = mx + b$$

parameters

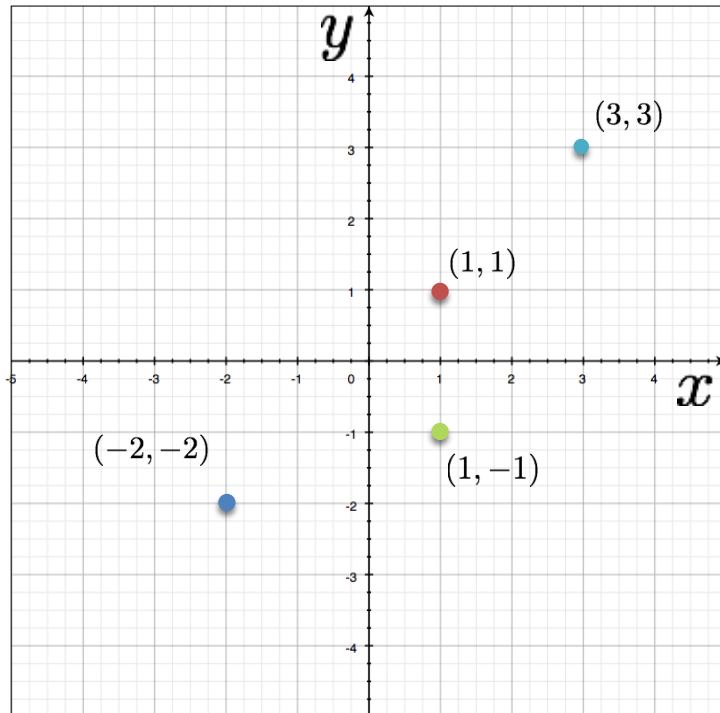


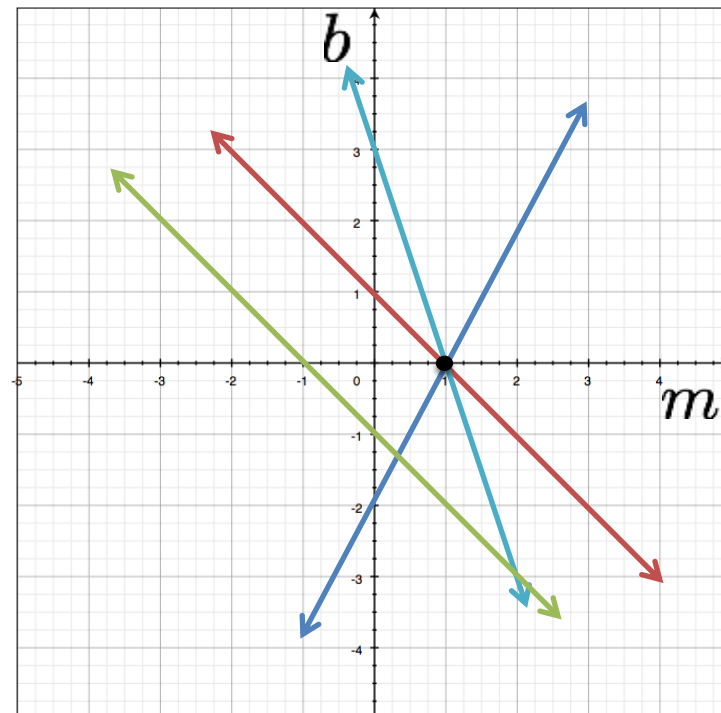
Image space

four points
become
?

variables

$$y - mx = b$$

parameters



Parameter space

How would you find the best fitting line?

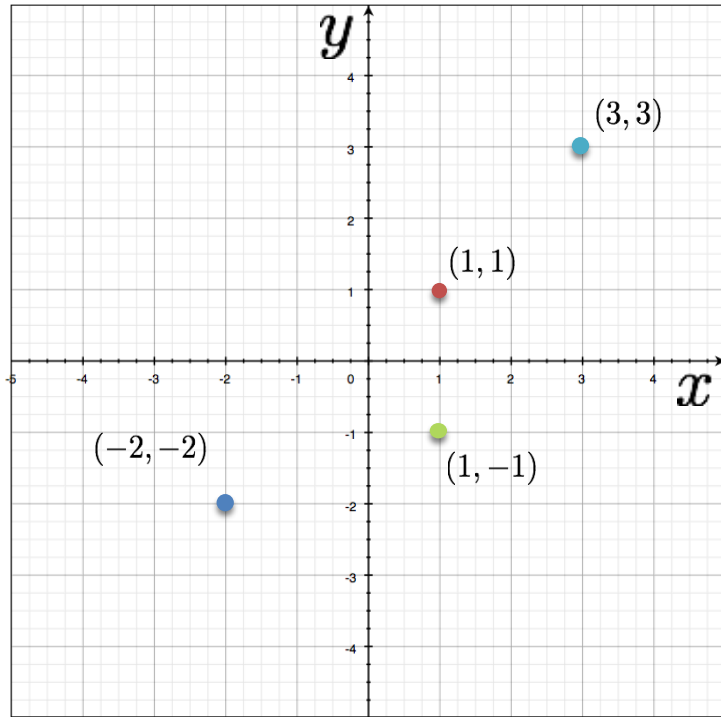
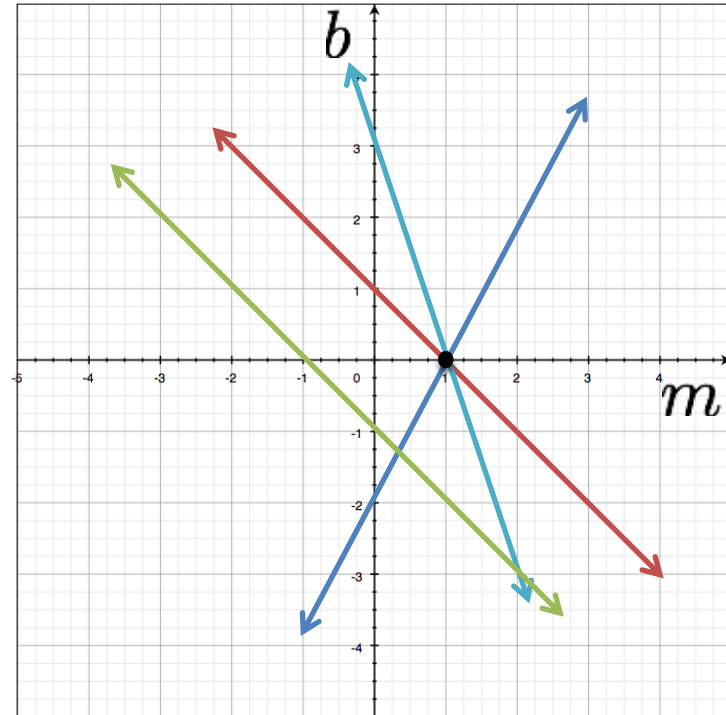


Image space



Parameter space

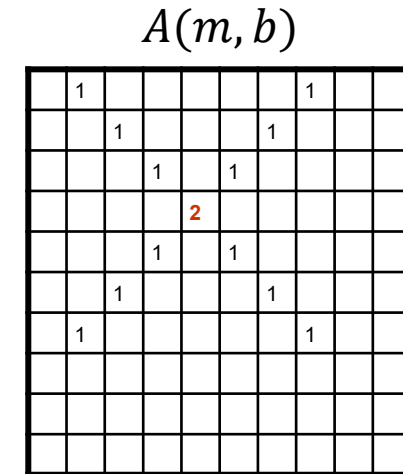
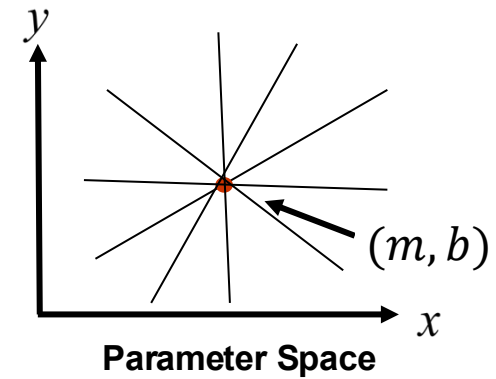
Is this method robust to measurement noise?

Is this method robust to outliers?

Line Detection by Hough Transform

Algorithm:

1. Quantize Parameter Space (m, b)
2. Create Accumulator Array $A(m, b)$
3. Set $A(m, b) = 0 \quad \forall_{m, b}$
4. For each image edge (x_i, y_i)
 - For each element in $A(m, b)$
 - If (m, b) lies on the line: $b = -x_i m + y_i$
 - Increment $A(m, b) = A(m, b) + 1$
5. Find local maxima in $A(m, b)$



Problems with parameterization

How big does the accumulator need to be for the parameterization (m, b) ?

$A(m, b)$

	1						1		
		1					1		
			1		1				
				2					
			1		1				
		1					1		
	1							1	

The space of m is huge!

$$-\infty \leq m \leq \infty$$

The space of b is huge!

$$-\infty \leq b \leq \infty$$

Better Parameterization

Use normal form:

$$x \cos \theta + y \sin \theta = \rho$$

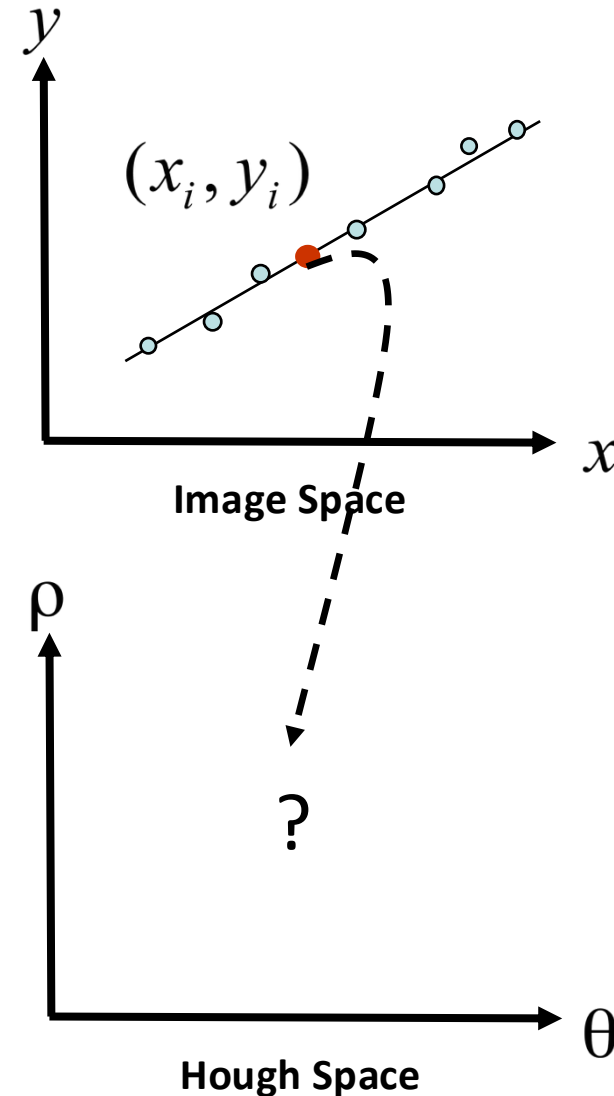
Given points (x_i, y_i) find (ρ, θ)

Hough Space Sinusoid

$$0 \leq \theta \leq 2\pi$$

$$0 \leq \rho \leq \rho_{\max}$$

(Finite Accumulator Array Size)



Implementation using Normal form

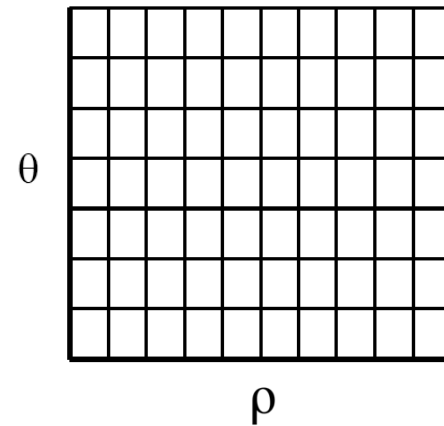
1. Initialize accumulator H to all zeros

2. For each edge point (x,y) in the image
 For $\theta = 0$ to 180
 $\rho = x \cos \theta + y \sin \theta$
 $H(\theta, \rho) = H(\theta, \rho) + 1$
 end
end

3. Find the value(s) of (θ, ρ) where $H(\theta, \rho)$ is a local maximum

4. The detected line in the image is given
by
 $\rho = x \cos \theta + y \sin \theta$

H: accumulator array (votes)



NOTE: Watch your coordinates. Image origin is top left!

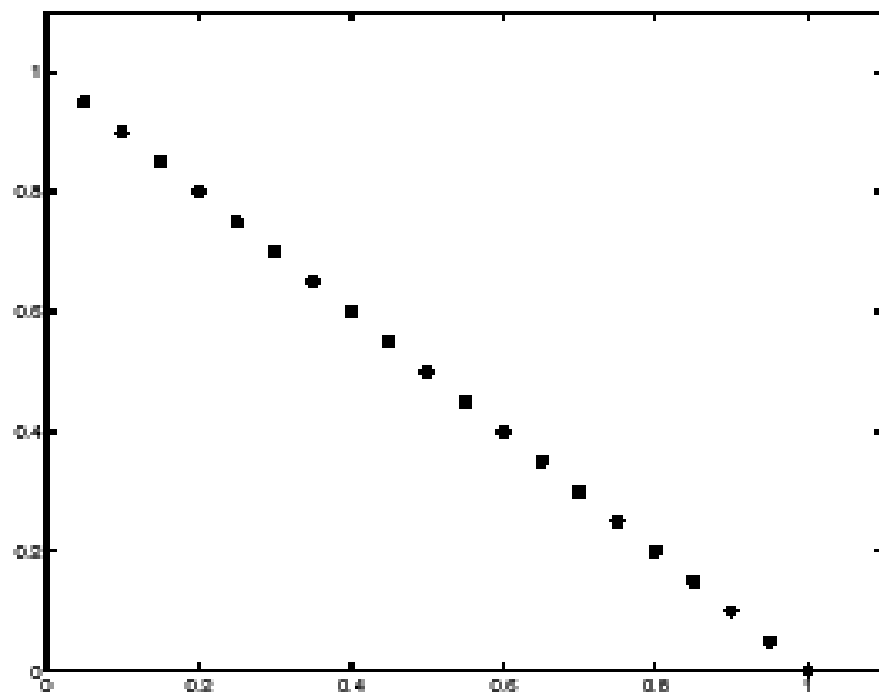
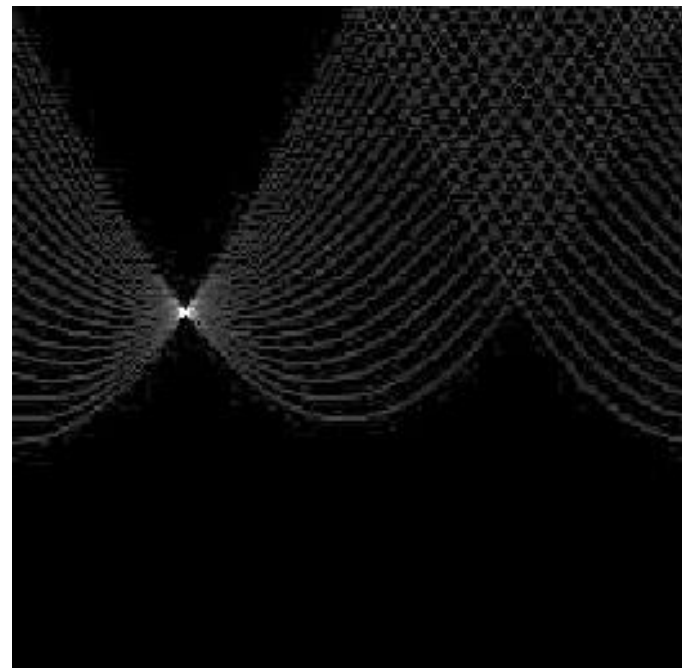


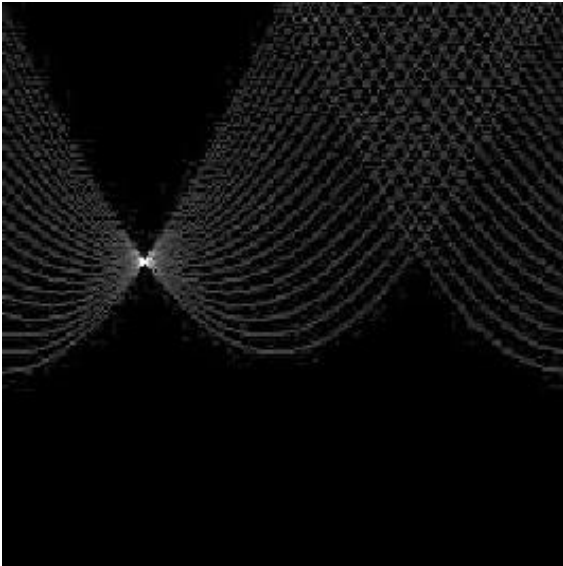
Image space



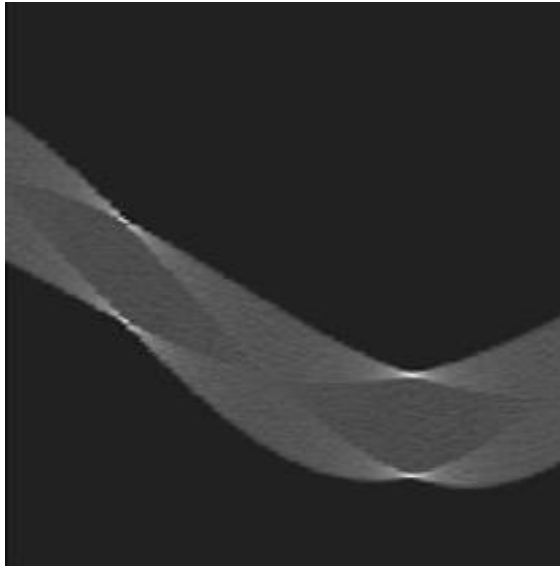
Votes

Exercise - Basic shapes

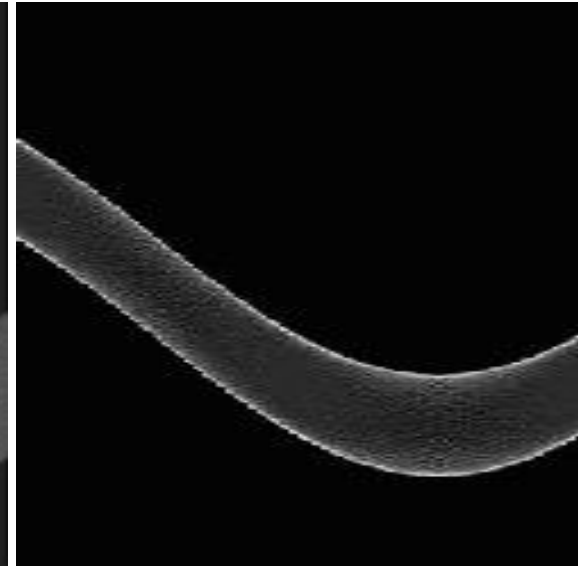
(in parameter space)



line

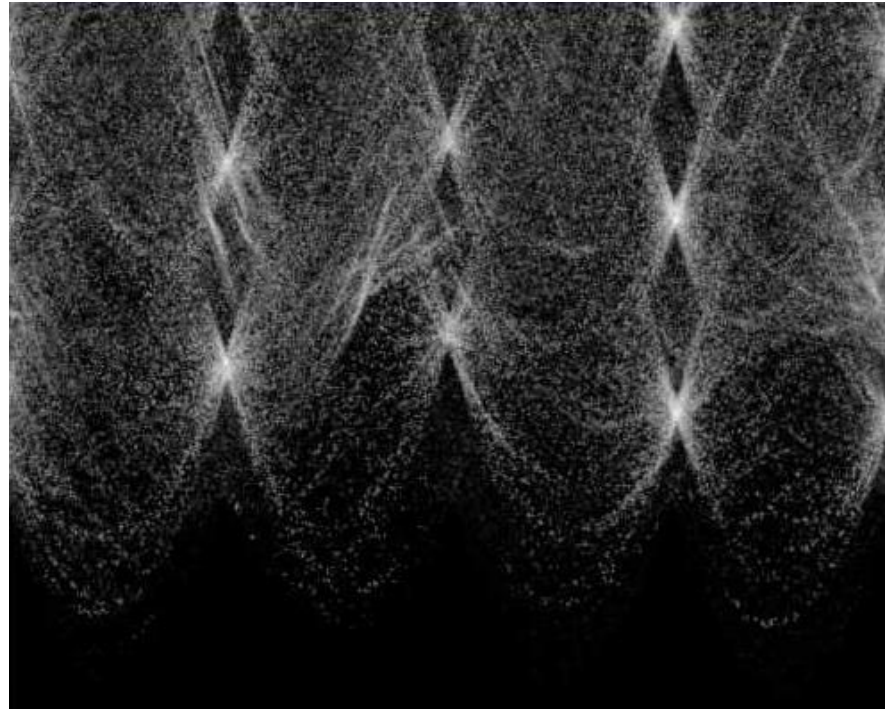


rectangle



circle

More complex image



In practice, measurements are noisy...

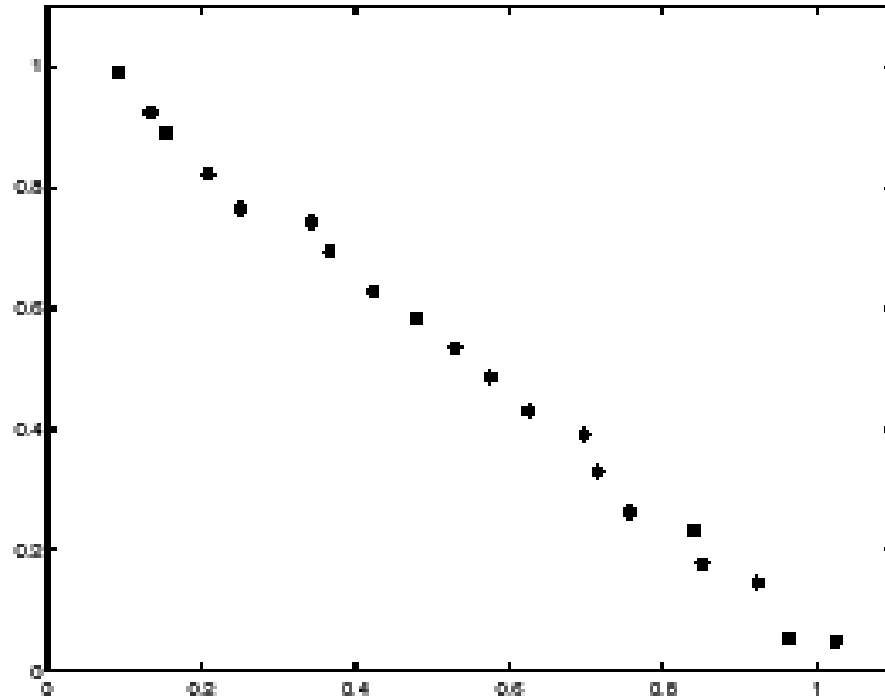
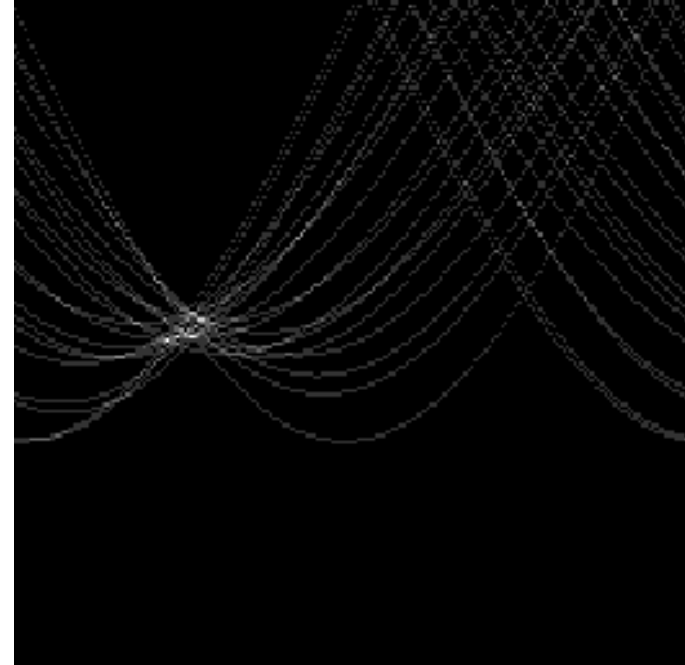


Image space



Votes

Too much noise ...

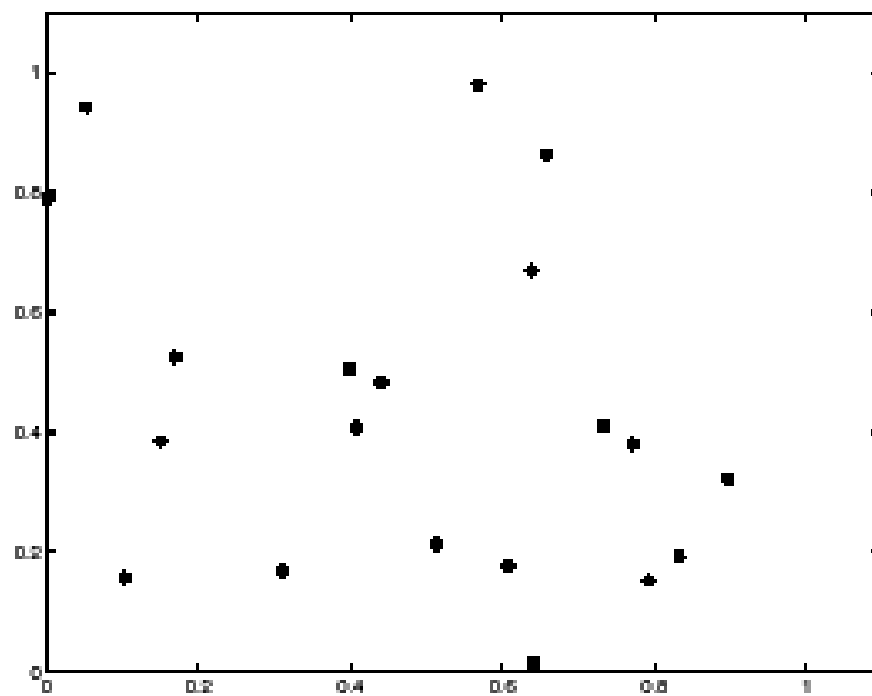
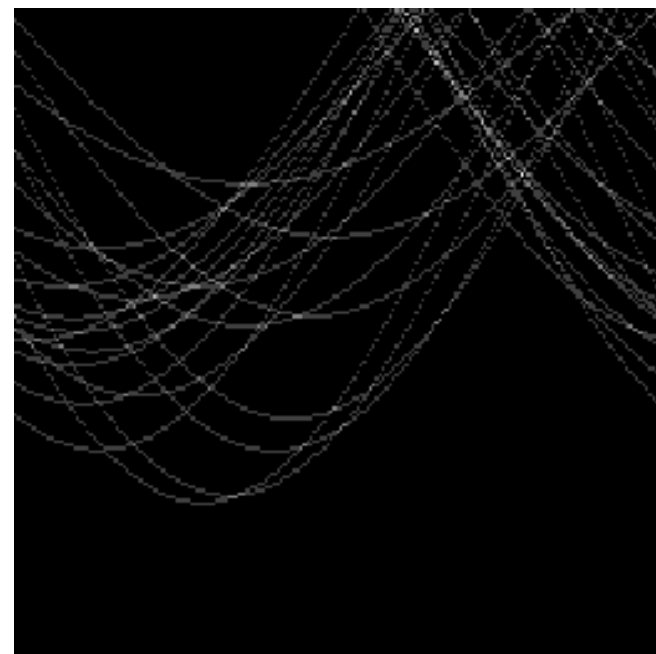


Image space

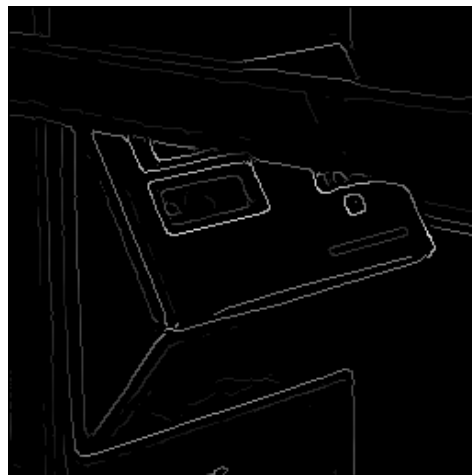


Votes

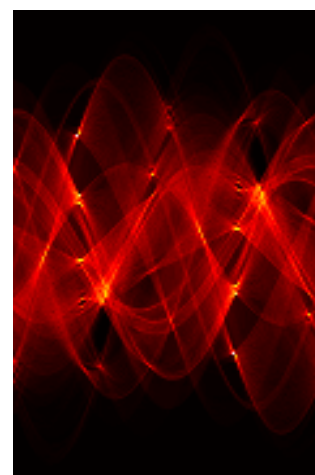
Real-world example



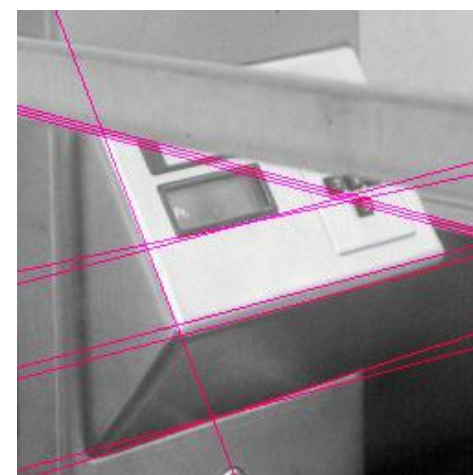
Original



Edges



parameter space



Hough Lines

Hough Circles

$$(x - a)^2 + (y - b)^2 = r^2$$

Diagram illustrating the Hough Circle equation. The variables x and y are indicated by green arrows pointing to the terms $(x - a)$ and $(y - b)$ respectively, with the label "variables" below. The parameters a and b are indicated by red arrows pointing to the terms a and b respectively, with the label "parameters" above.

- We can assume the radius is known
- Or, if radius is not known: 3D Hough Space!
- Use Accumulator array $A(a, b, r)$
- Surface shape in Hough space is complicated

